



Degree Project In Computer Science and Engineering

First Cycle, 15 credits

Voxel Cone Tracing Evaluation for Real-Time Applications

ALVAR TROJENBORG, FILIP NORTHMAN



Voxel Cone Tracing Evaluation for Real-Time Applications

ALVAR TROJENBORG
FILIP NORTHMAN

Degree Project in Computer Science and Engineering, First Cycle 15 credits

Date: June 10, 2024

Supervisor: Johan Jansson

Examiner: Pawel Herman

Swedish title: Voxel Konspårnings Utvärdering för Realtidsapplikationer

School of Electrical Engineering and Computer Science

Abstract

Voxel Cone Tracing is an approach that approximates global illumination for real-time applications and has some interesting properties. Global illumination gives photorealistic lighting results since it simulates how light in real life bounces around multiple times resulting in indirect lighting. Global illumination can be very expensive and a challenge for real-time applications, therefore approximation algorithms such as the Voxel Cone Tracing method make the computations easier. This method approximates the original scene and creates a voxel field to retrieve necessary radiance information during the sampling phase where indirect lighting is processed. Light is approximated by tracing cones that sample the voxel field, rendering realistic approximations with speed necessary for real-time usage. The Voxel Cone Tracing implementation in the report was implemented using C++ with the OpenGL graphics API. The Sponza model is rendered using Voxel Cone Tracing with sparse 3D textures. The qualities evaluated were performance, memory consumption, and visual accuracy, and their dependence on voxel resolution is studied. The qualities evaluated were performance, memory consumption, and visual accuracy. This way the method's strengths and weaknesses could be explored.

The thesis concludes that Voxel Cone Tracing is a method that warrants further study as advances in modern hardware have improved many of the inherent flaws of Voxel Cone Tracing, such as memory consumption and speed with higher accuracy renders. As for the visual quality the foremost unrealistic lighting artifacts were voxel approximation errors, especially around concavities and unwanted light bleeding from thin materials. The Visual quality was high besides these errors and there was space for further improvements and optimizations to the implementation presented in this thesis. Implementing a more sophisticated voxel storage solution can lead to reduced memory usage and enhanced performance. Ultimately, voxel cone tracing emerges as a promising technique that warrants further exploration.

Sammanfattning

Voxel Cone Tracing är en metod för att approximera global illumination för realtidsapplikationer med flera intressanta egenskaper. Global illumination skapar fotorealistiska ljusresultat då det simulerar hur ljuset i den verkliga världen studsar runt flera gånger, vilket resulterar i indirekt ljussättning. Global illumination kan vara mycket påkostat och utmanande för realtidsapplikationer, varför man använder approximerande algoritmer såsom Voxel konspårnings tekniken för att underlätta beräkningarna. Denna metod approximerar den ursprungliga scenen och skapar ett voxelfält efter det för att kunna hämta nödvändig ljusstrålningsinformation under provtagningsfasen där indirekt ljus behandlas. Då approximeras ljuset genom att spåra koner som provtar i voxelfältet vilket resulterar i rendering av realistiska approximationer av ljus med en hastighet tillräcklig för realtids användande. Implementationen i denna rapport använder C++ med OpenGL som grafik API. Sponzascenen renderas med Voxel Cone Tracing med olika 3D texturer. Resultatens olikheter beroende på texturerna studeras därefter. De olika egenskaperna som analyseras är prestanda, minnesförbrukning och visuell tillförlitlighet. På detta sätt kan metodens styrkor och svagheter utforskas.

Slutsatsen konstaterar att Voxel Cone Tracing är en metod som kräver vidare studier så framstegen i den moderna hårdvaruutvecklingen har botat många av metodens initiala brister, så som minnesförbrukning och hastighet med mer noggranna renderingar. Gällande visuell kvalitet var de mest orealistiska ljusartefakterna Voxel estimeringsfel, speciellt vid konkaviteter och oönskat ljusflöde genom tunna material. Den visuella kvalitén var, utöver dessa fel, hög. Det finns utöver detta utrymme för förbättringar och optimeringar för implementationen som presenterades i rapporten. Genom att implementera en mer sofistikerad Voxel lagringslösning kan det leda till en minskad minnesförbrukning och ökad prestanda. Sammanfattningsvis är Voxel Cone Tracing en lovande teknik som bör utforskas mer.

Contents

1 Terminology	1
2 Introduction	2
2.1 Problem Statement	3
2.2 Purpose	3
2.3 Limitations	4
2.4 Source code	4
3 Background	5
3.1 Introduction to Computer Graphics.....	5
3.1.1 Rendering	5
3.1.2 Real-time and offline rendering	5
3.1.3 The graphics pipeline	6
3.2 Global Illumination	8
3.2.1 Global illumination explained	8
3.2.2 Global Illumination problems	11
3.2.3 Global Illumination problem solutions and approximations.....	12
3.2.4 BRDF and approximation of the rendering equation	14
3.2.5 The Lambertian model for diffuse reflections.....	14
3.2.6 Specular ray trace and Phong model for specular reflections	15
3.3 Voxel Cone Tracing	16
3.3.1 Voxelization and storing direct lighting.....	16
3.3.2 Mipmapping	19
3.3.3 Cone tracing	20
3.3.4 Multiple light bounces.....	23
4 Method	25
4.1 Implementation.....	25
4.1.1 Setup and Hardware	25
4.1.2 Voxelization of the scene	25
4.1.3 Voxel Cone Tracing implementation	27
4.1.4 Final color output	28
4.2 Evaluation methods	29

5 Results	31
5.1 Performance Results.....	31
5.2 Visual Results.....	32
5.3 Memory Consumption of Voxel Textures	37
6 Conclusion and Future work	38
6.1 Discussion	38
6.1.1 How do different voxel sizes and light behavior affect performance?	39
6.1.2 How do different voxel sizes and light behavior affect memory consumption?.....	39
6.1.3 How do different voxel texture sizes and light behavior affect image quality?.....	40
6.2 Further improvements	41
6.3 Conclusion.....	41
7 Sources	42
Appendix	45

1 Terminology

AO: Ambient Occlusion

Scene: the virtual environment full of models/objects rendered

LOD: Level of details.

PBR: Physically Based Rendering

Voxel: Volumetric pixel, a cube in a 3-dimensional space that can contain 3D data like location, color and density.

Mesh: A collection of vertices, edges and faces defining the surface of an object.

Primitive triangle: Collection of vertices that form a single 3D entity, usually but not limited to a triangle.

OpenGL: Cross platform rendering API.

MSAA: Multi-sample anti-aliasing.

FBO: Frame Buffer Object

OpenGL: extension enabling off-screen rendering.

Texel: An element of a texture.

BRDF: Bidirectional Reflectance Distribution Function.

Radiance: Radiant Energy emitted, reflected or received by a given surface.

Render/Image Synthesis: Automated process of creating an image.

Aperture: Angle of the view cone.

Mipmap: Collection of image data with gradually lower level of detail.

Texture map: 2D image of what colors the object radiates upon lighting, that is the look of the object. Each coordinate on the texture map maps to a position on the belonging mesh object.

Sampling: The process of evaluating and gathering data values from a texture map or other storage structures used in the scene.

Denoiser: Methods to average out divergent pixel colors from bad sampling results.

Fragments: Potential final viewable pixels on screen.

High-frequency aliasing: Image artifacts, especially jagged edges at high frequency components, that is any area where the color changes abruptly, that happens if resolution is too low to accurately capture the rapid light changes.

2 Introduction

Producing realistic images and imitating reality is one of the focus points in the world of computer graphics. Therefore, it is important to understand how light works, both in the real world but also how to mimic it using computers. Simulating realistic light is done by combining several different types of light into one final result. The result is often called global illumination. Global illumination is the effect of when light spreads throughout a scene and creates realistic shadows. To get global illumination in real time applications, approximations are required to get the speed necessary for the program to be interactive. There are many ways to do this, but this thesis will examine one method, Voxel Cone Tracing. The Voxel Cone Tracing method was first published back in 2011 by Crassin et al. [1] and has until this day been used sporadically. For example, Voxel Cone Tracing made its way into the popular game engine Unreal Engine 4 under the name VXGI made by Nvidia, although the latest VXGI update was in 2018 [2]. However, it has never really dominated, as there are many other more prevalent alternatives for global illumination.

In short, Voxel Cone Tracing compromises and approximates the compute-heavy algorithm path tracing by tracing singular light photons. Voxel Cone Tracing instead traces cones covering many light particles at the same time. The cones are sent out from a point of interest to register the light radiated from its surroundings, thus providing the indirect illumination phenomena global illumination. All direct lighting contributes as indirect light and is stored in a voxel grid using a sparse 3D texture. Through Voxel Cone Tracing interactive speeds are achieved with realistic results, although with a few light artifacts.

2.1 Problem Statement

The aim of this thesis is to investigate the qualities of Voxel Cone Tracing, which can be narrowed down to its performance, memory usage and image quality. The thesis examines Voxel Cone Tracing's different qualities and how they change by altering voxel texture sizes and lighting behavior. Where light behavior can be defined as changing the number of light bounces and comparing static to dynamic scenes. A static scene is defined as a scene where the environment does not change, and a dynamic scene is a scene which changes each frame. The specific evaluation methods are further presented in Ch. (Chapter 4.2).

The thesis therefore aims to answer the following research questions:

Can an implementation of Voxel Cone Tracing render in real-time providing global illumination with sufficient speed, space consumption and image quality? How do different voxel texture sizes and light behavior affect these qualities?

The research question can be subdivided to more specific questions that explore the implementation's different settings and the effect on the result.

Q1) *How do different voxel texture sizes and light behavior affect performance?*

Q2) *How do different voxel texture sizes and light behavior affect memory consumption?*

Q3) *How do different voxel texture sizes and light behavior affect image quality?*

2.2 Purpose

The purpose of this thesis is to understand the qualities of Voxel Cone Tracing. It is interesting to change both the light behavior and voxel data structure size as they both affect the realism of the results seen in the image quality and the algorithm's performance and memory consumption. Balancing these three quality aspects is a core problem in computer graphics and ultimately defines how good a rendering method is. This thesis tries to pinpoint what qualities this Voxel Cone Tracing implementation falters or excels.

It is important to understand that there are multiple ways to implement a Voxel Cone Tracer and many optimizations and variations have come up over the years. After all, 13 years have passed since this method gained notable attention after Crassin et. al. first presented their paper “Interactive Indirect Illumination Using Voxel Cone Tracing” [1]. There are some improvements to the traditional Voxel Cone Tracing implementation presented by Crassin used in the implementation of the thesis evaluations. The improvements are the usage of physically based rendering techniques and multiple light bounces. The hypothesis is that these improvements create more realistic lighting.

2.3 Limitations

The implementation of Voxel Cone Tracing in this thesis has some limitations. Only one method of storing the voxel textures is implemented and evaluated. There are other methods for storing voxels that are equally promising and further comparisons for the thesis could improve the overall result. Another limitation is that the thesis only implements 1 and 2 light bounces for the Voxel Cone Tracer. It is possible to extend the method to calculate an indefinite number of bounces using the same memory consumption as 2 bounces. Further studies upon the visual results and performance of more bounces and other storage methods should be expanded upon in future papers.

Because of implementing from scratch and the aim of seeking deeper understanding of the inner workings of our implementation, the time constraints of the thesis created some compromises regarding the Voxel Cone Tracing implementation. The main compromise is the storage format of the voxels. This thesis uses a sparse 3D texture which is a good approach although not optimal with storage efficiency in mind. A better approach would be to implement a Clipmap[18][19] or a dynamic sparse voxel octree [1] which could handle dynamic lighting in scenes more effectively and use less memory, although.

2.4 Source code

The source code of the implementation uses C++ and OpenGL 4.6 is available at <https://github.com/FilipNorth/KEX-tracer/tree/Thesis-Result>. A video demonstrating the result is also available in the GitHub repository.

3 Background

3.1 Introduction to Computer Graphics

3.1.1 Rendering

Rendering is the process of generating an image from a model holding information using a computer program. The goal of rendering is to provide the user and viewer with the illusion of watching real objects on the screen of the device. The image on the screen consists of pixels, evenly distributed tiny colored rectangles. The color of the pixels is generated from a virtual world, a scene full of objects that contains a lot of information the computer uses to determine the pixel's colors. [3]

Since this thesis focuses on global illumination, realistic light behavior and the related rendering process is the focus instead of a stylistic one. In computer graphics, there are a myriad of different methodologies and techniques to render a scene. The area of use for each method can be generalized to the basic question of if the technique should be used for real-time or offline rendering.

3.1.2 Real-time and offline rendering

Offline rendering, while being able to render detailed photorealistic images (see figure 1) the output speed is very slow due to the huge number of resources it takes to create it.



Figure 1: *Offline rendered photography of a scene with many realistic lighting effects* [4].

While this works to render an animated movie that can take two whole years and about 50 hours per frame sometimes according to Pixar Animation Studios [14], it is not acceptable for real-time mediums such as video games. If rendering in real-time, the computer is required to output around 25-70 frames per second for a good and interactive user experience [1]. To manage this frame rate, approximations need to be made. The quality and power of the GPU, the main

computing force and bottleneck of global illumination algorithms, keeps on improving and at the same time new and more refined optimizations of ray tracing have been made. Recently a lot of promising papers and improvements, especially from NVIDIA [24][5][6], have been released. These papers keep on proving that the graphics industry is moving closer to a reliable real-time ray tracer supported global illumination implementation and the optimizations and development does not seem to slow down any time soon [22]. As these new promising technologies are still new and require specific hardware to use, they are not accessible to the general population. Other algorithms such as Voxel Cone Tracing which compromise on the accuracy of the global illumination in favor of speed are still excellent alternatives.

3.1.3 The graphics pipeline

Vertices hold different attribute information and almost always form in a group of three primitive triangles. These primitives and vertices make up the final textured 3D model in the virtual world that renders on the screen. The process of the GPU reading the vertex data and consequently rendering the 3D models on a 2D screen follows the graphics pipeline. This transformation is divided into different stages in the graphics pipeline [25][26]. Note that shaders are programs that run on the GPU instead of the CPU.

Vertex Shader

The Vertex shader handles the vertex attributes such as the position, texture coordinate and normal vector. Mainly the position coordinates are transformed so that they are positioned correctly in clip space coordinates where the origin is at the viewport, that is the origin of the camera, and perspective by matrix transformation is applied. Orthographic matrices needed for voxelization can also be applied here, meaning that perspective is omitted so that objects always retain their size regardless of their distance from the camera [25][26].

Geometry shader

The geometry shader can modify the primitive geometries or modify them to generate new geometries, which is also needed during the voxelization process. The geometry shader has the unique attribute that they handle each vertex of a triangle individually instead of the entire triangle at once [25][26].

Rasterization

The rasterization is a fixed function process that introduces a grid of pixels based on screen resolution and converts the primitive to *fragments* based on its pixel coverage, see Figure 2.

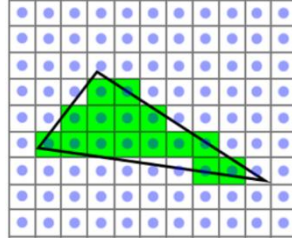


Figure 2: Rasterization of the triangle by checking if the middle point of the pixels overlaps with the triangle area [21].

The rasterizer then interpolates the vertex attributes across the primitive's surface, see Figure 3. This means that the information that the vertices at each corner are containing is being averaged at every fragment between the vertex fragments.

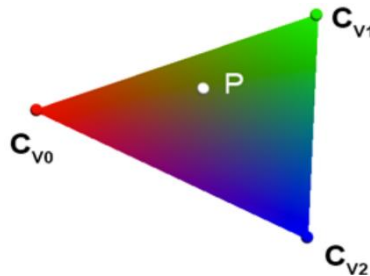


Figure 3: The colors C of vertices V_0 , V_1 and V_2 are interpolated, averaging the color at each point P inside the triangle [21].

By doing a depth test the GPU determines which fragments should be sent to the fragment shader to be viewable in front of the other occluded fragments [25][26].

Fragment shader

The fragment shader takes the fragment's interpolated attribute values and calculates and outputs the color of the fragment. This is done by sampling a base color from its corresponding texture maps using its interpolated texture coordinates and then applying lighting effects from the light sources in the scene [25][26].

Compute shader

This shader is not directly a part of the traditional graphics pipeline, but it is nevertheless needed to effectively compute large amounts of data that can then be used as buffers in the graphics pipeline. In Voxel Cone Tracing, the compute shader is very useful for calculating more than one light bounce in the scene [29].

3.2 Global Illumination

3.2.1 Global illumination explained

Global illumination is a name for all methods allowing light to bounce around in the scene, which creates indirect light. Global illumination creates the effect of realistic lighting in a scene and thus greatly enhances the realism of a rendered scene. Without global illumination, areas in a scene that are not directly lit from a light source would be completely black [4]. However, in the real world these areas would just be darker. To simulate this effect, the primary method involves allowing light to bounce off surfaces to indirectly light up areas, see figure 4.

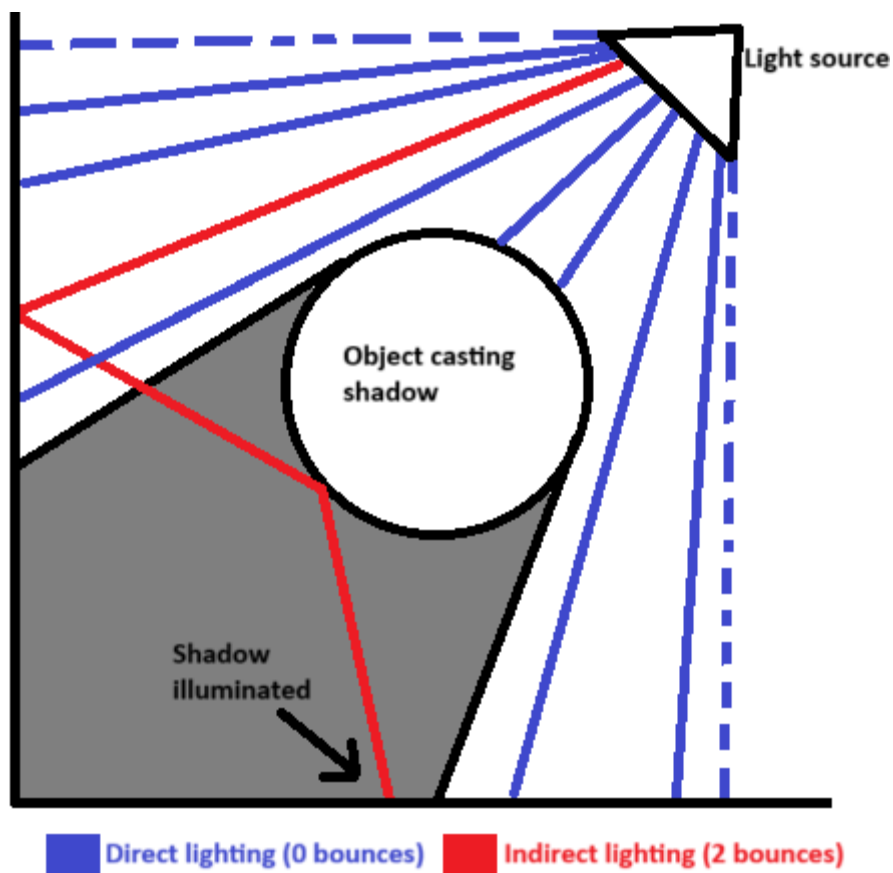


Figure 4: *Illustrating shadow illuminated by the indirect lighting.*

To better visualize global illumination and some of the different aspects of it the different parts of global illumination become visualized in figure 5, which expands figure 1 from Ch 3.1.2

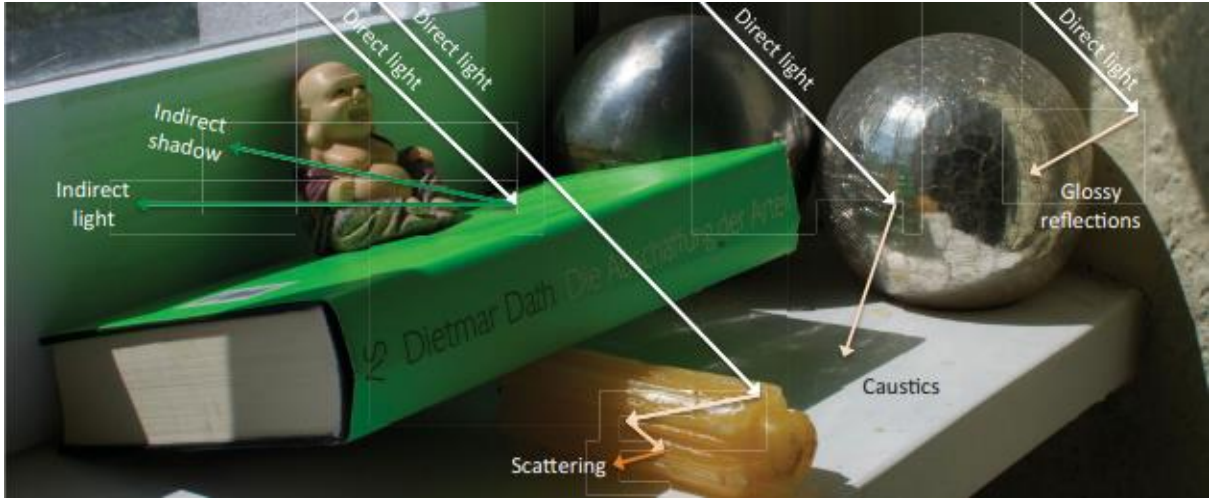


Figure 5: *global illumination effects in the scene are specular and diffuse light bounces, scattering and caustics [4].*

This report mainly focuses on the simpler aspects of global illumination that is diffuse, specular and direct light. Color bleeding will also be a by-product based on the object surface properties which makes the reflected light propagate the color of the object it was reflected on to its surroundings.

To understand global illumination better a closer look at the rendering equation written by Jim Kajiya in 1986 is required [7]. This equation to this day still lays the foundation of all the rendering done as it accurately describes how light is transported and how it interacts with surfaces in the real world. It acts as a sort of unified theoretical framework for all different kinds of global illumination algorithms. This rendering equation is used as a basis in the fragment shader when deciding the final output color of a fragment. Equation 1 (*Eq1*) is a more readable and simplified formulation of the rendering equation. Notice that variables denoted with i stands for incoming (light) and o stands for outgoing (light):

$$L_o(\omega_o) = \int_{\Omega} L_i(\omega_i) \cos(\Theta_i) f_r(\omega_i, \omega_o) d\omega_i$$

Eq1: A more readable and simplified version of the rendering equation from Kajiya [7].

Regard a point of interest p on a surface where light will bounce. The light interaction at p can be calculated with *Eq1*. The $L_i(\omega_i) \cos(\theta_i)$ product is for the effect of incoming light to p , calculated by multiplying two factors. The first factor $L_i(\omega_i)$ is the amount of received light at p originated from the light source L_i that sent light in direction ω_i directly towards p . It is critical that this incoming light can represent both direct and indirect light. The second term is the geometry term $\cos(\theta_i)$ where θ_i is the angle created from θ_i and the surface normal N of p thus scaling the lights effect depending on the incidence angle θ_i . Notice that $\cos^+(\theta_i)$ is used

and that $0 \leq \cos(\theta_i) \leq 1$. It is often calculated and written as the positive dot product with the normalized vectors $\omega_i \cdot N$ [7].

The $f_r(\omega_i, \omega_o)$ function is the bidirectional reflectance distribution function (BRDF) which decides the statistical distribution of how light bounces off an object and thus how the object is perceived by the viewer [7]. Both the color and mirroring abilities of an object, however, depend on the material properties making the material and BRDF closely linked together. The material properties are based on the objects' PBR textures that simulate the surface's base color, metallicity- and roughness properties. How the light exactly will interact with the PBR textures are calculated with the BRDF. BRDF calculates the proportion of the light from the incoming light direction i and the light to the outgoing light direction ω_o . So all the reflected outgoing light in direction ω_o , that is $L_o(\omega_o)$, given from a single light source is given by the equation $L_i(\omega_i) \cos^+(\theta_i) f_r(\omega_i, \omega_o)$, see Figure 6.

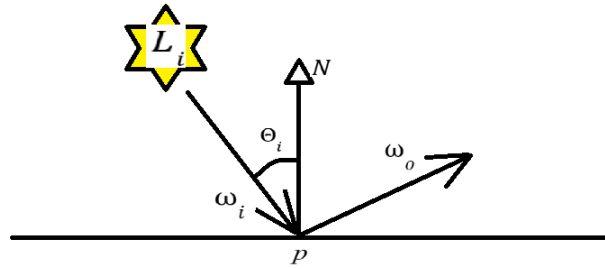


Figure 6: Illustrating variables from the rendering equation relations around surface point p .

Important to understand is that when rendering it is assumed that $L_o(\omega_o)$ is towards the viewer or camera so that the calculations result in what the camera can see.

However, at the surface point p the light is coming from all directions, meaning an infinite number of light directions hit the surface. The light is coming from all over a hemisphere Ω centered around p , see Figure 7. Notice that this is the positive hemisphere Ω^+ with direction of the normal N . This can be done by calculating an integral over the entire Ω^+ surface. For each $L_i(\omega_i)$ in every direction over Ω^+ , $L_o(\omega_o)$ is calculated and then by the property of light addition is used to accumulate the linear additive output light to a final viewable $L_o(\omega_o)$ [7].

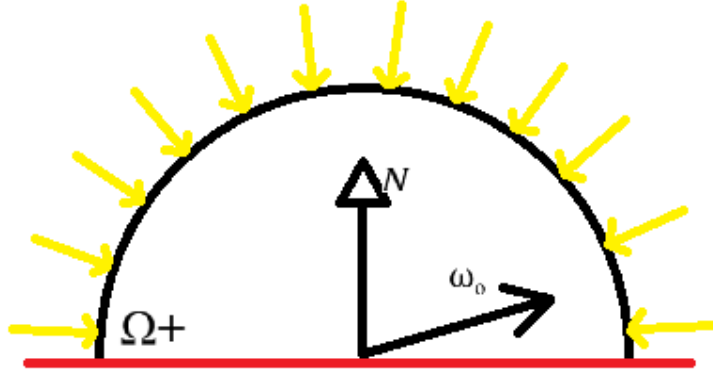


Figure 7: The yellow arrows represent how light coming from every direction over Ω^+ is contributing to $L_o(\omega_o)$.

To be correct, according to Kajiya a constant term $L_e(\omega_o)$ should also be added that represents the light emitted from point p itself finally giving us the real rendering equation Eq2, still in a more readable way:

$$L_o(\omega_o) = L_e(\omega_o) + \int_{\Omega} L_i(\omega_i) \cos(\Theta_i) f_r(\omega_i, \omega_o) d\omega_i$$

Eq2: A more readable but complete version of the rendering equation from Kajiya [7].

However, in the Sponza scene there is no light emitting object except for explicit light sources L_i , so $L_e(\omega_o)$ can be ignored in this case.

3.2.2 Global Illumination problems

Implementing and solving the rendering equation accurately would prove to be very hard, mainly because of two reasons that will be referenced as *a)* and *b)* [4]:

- a. The hemisphere that is needed to closely model real-world lighting is computationally intensive to calculate an integral over every single incoming light direction and do this for each fragment point on the many surfaces that are rendered. The number of computations that would be needed for this are simply too many and not a feasible way to calculate the global illumination, at least in real time.
- b. The other problem is regarding the equation's recursive nature. The number of light bounces is indefinite, and each bounce affects its surroundings. So, for each surface point p , it is affected by the light bounces from the surrounding surface points n , and in turn this goes the other way. The emitted light from p will affect the other surrounding points n . Meaning that the lighting at p and n depend on each other. With two unknown

variables and without knowing the number of times bounced before the rendering equation cannot be solved analytically directly. In other words, the output of the equation is also the input of the equation, just at another point, see Figure 8.

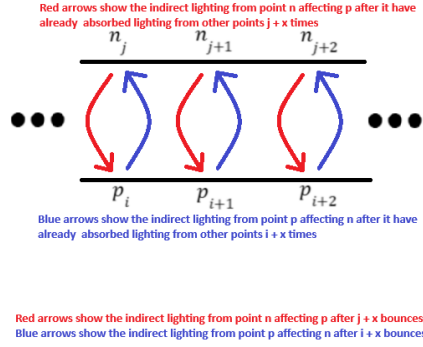


Figure 8: Illustrates how each surface point affects each other by indirect lighting multiple times.

3.2.3 Global Illumination problem solutions and approximations

The rendering equation is often more of a goal that instructs what should be approximated to get realistic lighting effects rather than a proposed solution. Approximations or other models can be used to simplify or adapt the rendering equation. This way computational efficiency can be assured, and real-time rendering is possible while still providing realistic lighting with global illumination [4].

In respect of rendering equation problem, *a*), the Monte Carlo Integration solves this by discretizing the continuous space over Ω to get an approximated computable solution. This is also what is happening when using path tracing. More precisely by using this integration, one could trace a specified finite amount of incoming light rays that are set in random directions over Ω . By tracing back from where the light came from, other surfaces or light sources are reached and by *sampling* the lighting information that is stored it will simulate how the sampled light affected the tracing start point. Then the same light ray can potentially continue to be traced by again bouncing and sampling in a new direction simulating multiple light bounces. Finally, all light information sampled from the traced light rays are summarized. With this approximation of Ω change *Eq1* to a new approximation of the rendering equation, *Eq3* [4][7]:

$$L_0(\omega_0) = \sum_i L_i(\omega_i) \cos^+(\theta_i) f_i(\omega_i, \omega_0)$$

Eq3: Rendering equation over a finite discrete i amount of incoming light directions.

Respecting rendering equation problem *b*), the first part of the solution is to simply set a finite amount of allowed light bounces. This way, for each surface point p , light is sampled using Monte Carlo integration and then traced. The sampled light rays gather light information for each bounce iteration until maximum allowed light bounces are reached or if the ray leaves the scene. A ray leaves the scene if a max trace distance is reached without reaching the surface [4].

The other part of solving problem *b*), is the order of calculation. It is desirable to have the light bounce the same number of times in the scene so that some points do not sample more than others leading to inaccurate lighting. If each point samples the same number of times, during the i -th light sampling precisely simulates the $(i-1)$ -th indirect bounce, as it is known that the sampled point provides the correct light data. For that reason, a computation order is defined when rendering. The main with a clear computation order is that every point can bounce once per iteration and then sample from its surroundings. The Monte Carlo integration method is used to update its stored light data. Although every point should only update its data after all points have finished sampling and before starting the next iteration. This way each point is sampling the same number of times, and it is known how many times a point has already absorbed light as the light bounces are calculated one bounce at a time [4].

Figure 9 illustrates that the first indirect illumination affecting p_i that is *sampled* in iteration 2 from its surroundings is direct lighting stored at the points from the direct lighting calculation from iteration 1. Then during iteration 3, all surface points p_i will sample the now accumulated lighting, that is the direct lighting with the indirect lighting from iteration 2. This means that the lighting data is updated for each surface point before calculating how much the next lighting bounce will affect the scene. It is important to note that when all lighting bounces iterations are complete the final colors are output before restarting at iteration step 1 again for the next frame, thus resetting the accumulated lighting data.

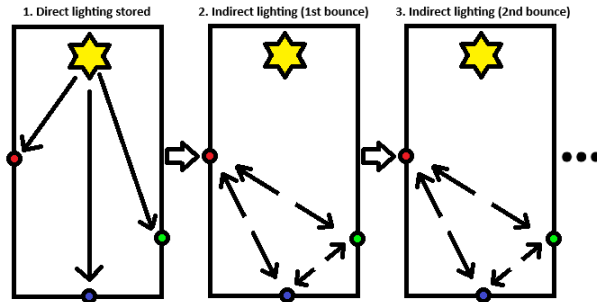


Figure 9: Shows how lighting data is updated during the first 3 lighting bounce iteration steps for three different example surface points.

This will simulate global illumination since both direct and indirect lighting is considered as light will bounce around in the scene. To calculate how much the total accumulated light $l(n)$ is at a point after light has bounced n times Eq4 is used. dL is direct lighting and iL_n is the indirect lighting calculated after n amount of indirect light bounces [4].

$$l(n) = dL + iL_1 + iL_2 + iL_3 + \dots + iL_{n-1}$$

Eq4: Light is accumulated from direct and each indirect light bounce affecting the point

3.2.4 BRDF and approximation of the rendering equation

Another thing to consider when approximating the rendering equation is how to formulate the Bidirectional Reflectance Distribution Function, BRDF. The BRDF oversees calculating how material responds to lighting based on the surface materials properties.

Choosing a BRDF all comes down to what materials there are and how much information about them is available in the scene. For example, there could be very complex materials with characteristics not covered in this report like anisotropic reflections or subsurface scattering requiring a different and more advanced BRDF [10][11]. Considering what information, the objects and their materials in the Sponza scene provide, a simpler BRDF will suffice for this Voxel Cone Tracing implementation.

BRDF changes depending on if the reflection is diffuse or specular as a Lambertian model is used for both direct and indirect diffuse reflections. For indirect specular reflections Voxel Cone Tracing is used while a Phong model is used for the direct specular reflections. This way the rendering equation can be divided depending on if $L_i(\omega_i)$ is from direct/indirect specular or direct/indirect diffuse lighting. Two different main lighting calculations can be done. One for *diffuse reflections* and one for *specular reflection*, both considering the direct and indirect lighting for a specified number of bounces as in Eq4. The results of both calculations can be added together for the final outputted fragment color $L_0(\omega_0)$ and thus approximate the rendering equation with Eq5 [4][7][13]:

$$\text{final output color} = \text{diffuse reflections} + \text{specular reflections}$$

Eq5: All the direct and indirect diffuse and specular light reflections contribute to final color

3.2.5 The Lambertian model for diffuse reflections

This is the most basic model for generating images that does not depend on the viewpoint coordinates where the camera is. It simply uses the surface normal, light source direction and the *albedo* color of each surface [12]. Where albedo is the percentage of light a surface reflects, hence determining the color of the object seen. In computer graphics albedo is often described as the texture map that defines the base color of an object before any light or shadows or other rendering effects are applied. The fact the light reflected is not dependent on viewpoint means that the same amount of light is reflected with the same amount albedo in every direction around the hemisphere Ω . This is called diffuse reflections, and they reflect light after some absorption in different directions depending on the uneven surface and is typical for matte and rough materials, see the right illustration of Figure 11. The different directions are approximated according to Monte Carlo Integration.

The rendering equation is approximated with Eq3, where the light hitting the surface point $L_i(\omega_i)$ is for direct light a coefficient $0 \leq L_i \leq 1$, depending on how bright the light source is, and for indirect light it is the stored lighting data from the other surfaces affecting our point of interest, see Figure 10. The BRDF $f_r(\omega_i, \omega_0)$ is the albedo color from the texture map for direct diffuse light while the sampled indirect diffuse light is scaled by the PBR roughness texture. In Figure 10 the many arrows from the surfaces represent the diffuse scattering and the arrows are

color coded as; red-indirect lighting, blue-reflected direct lighting, black-light source direct lighting. Also dotted arrows are just to clarify where they land, lighting is not scaled by the arrow's length [4][7].

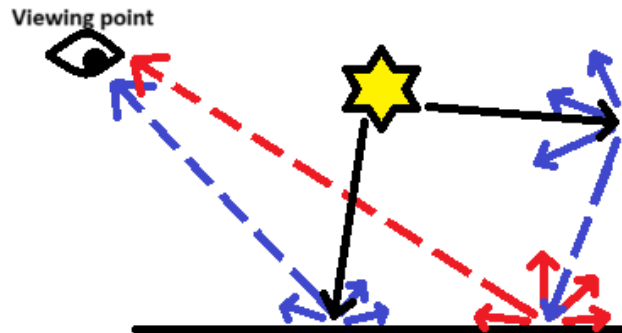


Figure 10: illustrating the diffuse lighting ending up at the viewing point from direct- or indirect light.

3.2.6 Specular ray trace and Phong model for specular reflections

Specular reflection defines that most of the light is reflected in a way so that the angle of incidence matches the angle of reflection, see the left illustration of Figure 11. It gives materials their shininess and appears if the material is smooth enough, giving it mirror-like properties. The specular reflections color that are seen is dependent on the viewpoint since ω_0 and ω_i need to be equiangular with the surface normal [7].

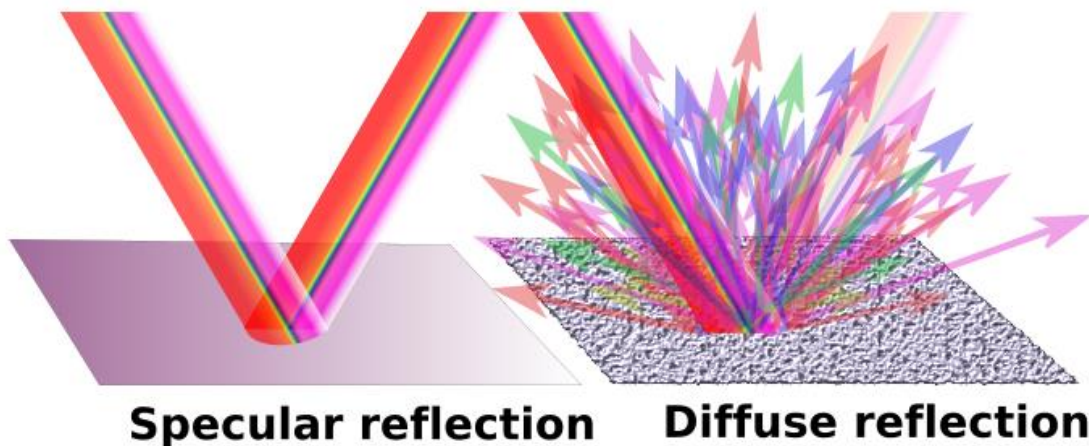


Figure 11, showcasing how light can either reflect in a specular or diffuse manner depending on material [20].

The rendering equation is used as a basis, however changing the Monte Carlo integration to instead sample specular rays in roughly the reflected view direction ω_0 . How tight the sample spread in direction of reflected ω_0 depends on the materials smoothness. A smoother material such as a mirror has a low spread while an uneven material such as a brick spreads light over the area [7][13].

The BRDF $f_r(\omega_i, \omega_o)$ for the direct light is based and scales with the metallicness of the PBR texture map while the indirect light is affected by the roughness of the PBR texture. The specular lighting hitting the surface point $L_i(\omega_i)$ is for direct light a coefficient $0 \leq L_i \leq 1$. For direct specular lighting a Phong model is used that transforms the geometry term to instead calculate the dot product of the reflected direct light direction ω_i with the viewpoint direction ω_o [27]. This gives surfaces a shininess effect that scales with how much ω_o and reflected ω_i line up.

For indirect specular light, $L_i(\omega_i)$ is the stored lighting data from the surfaces sampled from ray traces that mirrors the viewing angle and this gives surfaces a mirror-like effect. For indirect specular light the geometry term is omitted since the mirror effect of specular lighting is approximated to mirror lighting the same amount regardless of the incidence angle [4][7].

The illustration Figure 12 is read the same way as Figure 10, but here the identical angles θ_i and θ_j also showcase that the angle of reflection matches the incident angle.

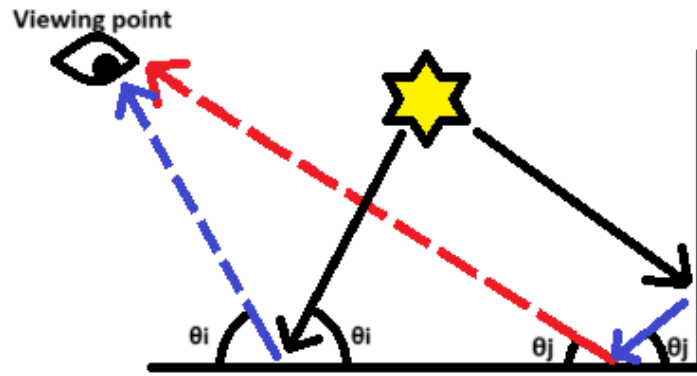


Figure 12: illustrating the specular lighting ending up at the viewing point from direct- or indirect light.

3.3 Voxel Cone Tracing

3.3.1 Voxelization and storing direct lighting

The solution for the rendering equation problem *b)* has already been discussed as seen in Ch 3.2.3, such that the direct lighting is calculated first and will contribute to the calculations of the indirect lighting bounces that follow. It will prove useful to store the direct lighting in a smart way so that the calculation of the indirect lighting will be more effective. Better still, this direct lighting is to be averaged and stored more roughly so that the “rough direct lighting” allows for quicker and larger data samplings. The rough direct lighting is only used to contribute to indirect lighting while the true direct lighting seen in the final rendered image is still accurately calculated for all fragments viewable on the screen space. The rough direct light is still calculated across the entire scene since the indirect light is emitted from surfaces even though they are not seen. This is opposed to the direct light that is only calculated if the surfaces are in the camera's field of view and not occluded. If the lighting is static the rough direct lighting can also be precomputed and reused, saving computation time later [1].

To store the rough direct light information, voxels are used. A grid full of voxels is created and the data from the scene full of triangle meshes are converted to the representing voxels, see Figure 13. Then the voxels data are put in a voxel data structure. All this using GPU shaders, and under the collective name voxelization. This means that the direct light of a scene is stored into our voxels to be used later for calculating indirect lightning [1].

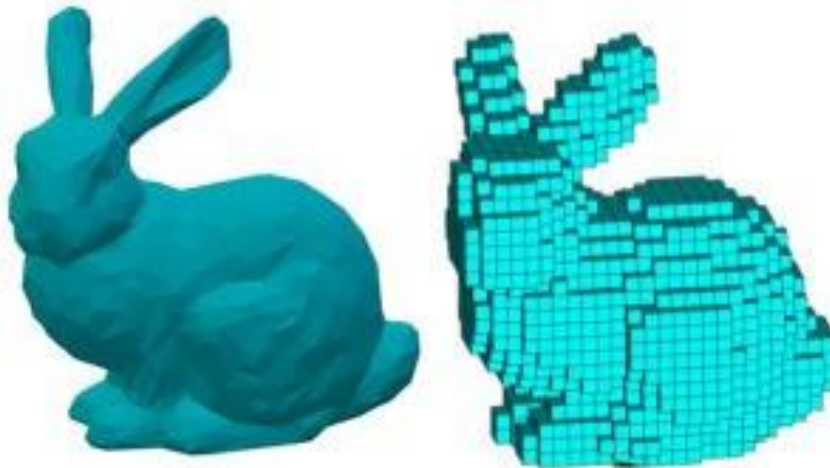


Figure 13: *Voxelization results of a bunny mesh. [23]*

Voxelization can be divided into three steps according to an article from NVIDIA Developer [15].

The first step is to use the vertex shader to transform the vertex positions into the voxel grid coordinate system. For this to work, it is important to apply an orthographic projection matrix in the vertex shader. The orthogonal projection given by the matrix should capture the entire scene in the visible clip-space, viewable from the camera. This means that the area to be voxelized is transformed so that it fits into a 3D texture where the voxels will be stored for later use. Since the entire scene needs to be voxelized, it is important that all primitives, even if occluded, are handled in the fragment shader. Therefore, before voxelizing the scene all depth testing needs to be disabled. Afterwards the geometry shader is used to project every primitive on the same 2D surface, temporarily substituting the correct depth value [1][15].

Step two, during rasterization of the transformed primitives, the rasterized fragment position together with a belonging correct depth value will exactly match its corresponding voxel grid coordinate. This means that this fragment data can be directly saved into the voxel grid data structure using the fragment coordinates from the shader. For this to work, it is important to make sure that the chosen viewport resolution exactly matches the chosen voxel grid scale, meaning the size of a fragment matches the size of a voxel. Only then, together with the orthogonal viewport frustum, the coordinates will match [15].

Finally step three, in the fragment shader. For each primitive the direct light information is calculated and stored in the chosen voxel grid data structure instead of writing it to the 2D conventional render target screen. The direct light computation is using the previously discussed Lambertian model [15].

The data is saved in a sparse 3D texture. This sparse 3D texture's texels (minimum unit of texture) stores the color information from the direct lighting. This data structure is a sparse voxel data structure, meaning that areas without voxels will not take up space in the texture. It is implemented by using the "GL_ARB_sparse_texture" extension in OpenGL. This extension allows a 3D texture to be defined as a sparse texture. The implication of this is that chunks of the texture can be enabled or disabled, where an enabled chunk can save data while a disabled chunk takes no memory. Areas where there are voxels are enabled consume memory even in the spaces where there are no voxels saved, so it is important to not enable too much of the texture. Although only the areas with voxels could be enabled it would be very slow to do as sparse 3D textures are not optimized for very small chunks allocated [30].

The voxels themselves can affect memory consumption even more if anisotropic voxels are used instead of isotropic. Anisotropic voxels have different data for each voxel face; hence the memory usage is increased sixfold for the 3D texture data structure compared to isotropic voxels that share data for all faces. On the other hand, having six different faces allows for more accurate illumination and avoids problems such as when the light is only on one side but will emit on all sides. This is not normally a problem as most voxels are only visible where the light hits the voxel, but for some thin objects light could hit one side but both sides are lit up without anisotropic voxels [1].

Often cracks form in between the voxels, see Figure 14 [15].

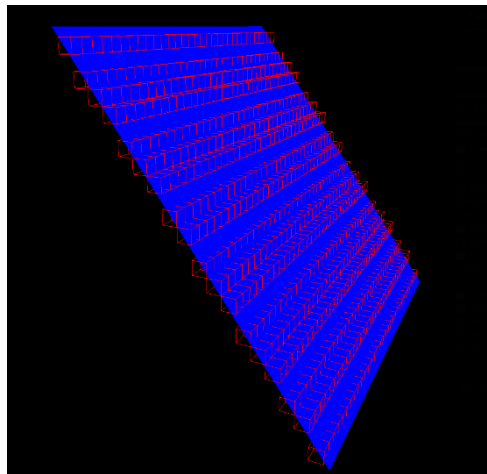


Figure 14: *Voxelization cracks between voxels.* [15]

The geometry shader will prevent this by projecting the primitives on a 2D surface in a smart manner. Based on the axis that presents the largest face area that corresponds to its greatest "axis shadow" when looking through the camera perspective along one of the axes, the primitive is projected on the 2D surface ready for rasterization and maximizing its area [15], see Figure 15 and Figure 16.

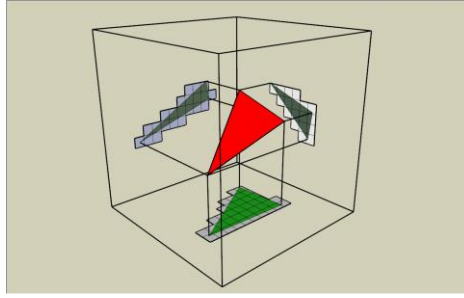


Figure 15: *Three potential axis shadows of a primitive to be projected.* [15]

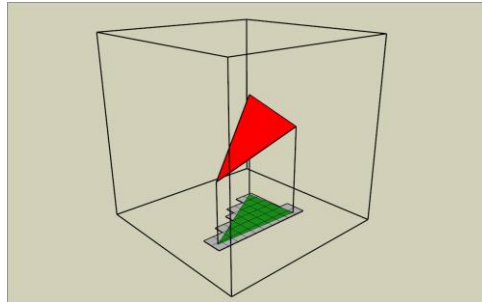


Figure 16: *Y axis provides the largest face area and is used for rasterization.* [15]

By swizzling the X, Y and Z components and substituting the depth to zero the primitive triangle is projected on the 2D surface ready for rasterization and the fragment shader to voxelize it.

Even so, gaps can still appear, where the rendered geometry does not fill the voxel fragments fully. To fix this hardware conservative rasterization is performed by enabling the “GL_CONSERVATIVE_RASTERIZATION_NV” extension in OpenGL when performing the voxelization. The main idea with this rasterization method is to expand the outer edges of the triangle primitive. This way a fragment should be considered covered if any part of it is touched by a primitive. Conservative rasterization is important when voxelizing the scene to minimize errors in later stages [15][31].

3.3.2 Mipmapping

Mipmapping is also referenced as filtering when down sampling 3D textures. Down sampling is used to compute an average value of a region of space in a 3D texture. For example, the average radiance spreading throughout a region of the scene. The down sampling filter is a $2 \times 2 \times 2$ kernel that averages the values and stores the result in a separate lower resolution 3D texture in a process called mipmapping. Storing these extra lower resolution textures results in an extra memory consumption of roughly 33% [18]. By down filtering the stored voxel radiance, *high frequency aliasing* from sampling is reduced since averaged light information is sampled compared to high frequency components. Shadows and corners that traditionally provide high frequency details and aliasing are thus smoothened. Generation of mipmapped textures is widely supported on modern GPUs, except for 3D textures. This means that to generate mipmaps for a sparse 3D texture it needs to be implemented using a compute shader to do the same thing. The difference in qualities between the filtered 3D textures can be described as a difference in the level of details, LOD [32].

3.3.3 Cone tracing

Cone tracing can be seen as a method that approximates both Monte Carlo integration and path tracing, all in a way that respects and satisfies the rendering equation. Different cone apertures and formations provide different lighting approximations. Important to note, is that the fragment shader is shading what is visible in clip space and using depth test together with applying perspective compared to the voxelization process. For each visible fragment in the fragment shader the indirect and direct diffuse light is combined with specular direct and indirect specular light in a way that approximates the rendering equation, like in Eq5. The four different reflection types and how they relate to cone tracing will be discussed now [1].

$$\text{final output color} = \text{diffuse reflections} + \text{specular reflections} \quad (\text{Eq5.})$$

Direct Diffuse Illumination

Direct diffuse illumination is calculated with the same Lambertian model covered in Ch 3.2.5 and it is the same calculation as when pre-computing the injected direct lighting into the voxels during voxelization but without mipmapping. No cones are used for this [13].

Indirect Diffuse Illumination

Indirect diffuse illumination is sampled by sending out cones from every surface in a half sphere, see figure 17, and collecting the light stored in the voxels in the scene. A few large cones will be traced and approximate the incoming indirect light all over the hemisphere Ω . This is reminiscent of the rendering equation problem solution a) discussed in Ch 3.2.2 that uses path tracing and Monte Carlo integration, to sample from discrete light directions as in Eq2. But instead of sending out rays at random there are predetermined cones that are traced that are fixed and symmetrically spaced. This ensures that the cones almost entirely cover the hemisphere Ω . All the different cone sampled indirect light sums are then accumulated as the total indirect diffuse light and are scaled. To approximate the rendering equation geometry term the surface and normal perpendicular cone has some additional weight to it. Notice that the total weights distributed amongst the cones are still 1. This approximation of the geometry term is supported by the fact that the cones are a coarse approximation with high apertures making it harder to pinpoint the incidence angle [1].

Direct Specular Illumination

Direct specular illumination giving shininess is calculated with the same Phong model covered in Ch 3.2.6. No cones are used for this [13].

Indirect Specular Illumination

Indirect specular illumination is sampled using a smaller specular cone in the viewing direction reflected about the normal. By using a cone instead of specular rays as discussed in Ch 3.2.6, the angular aperture of the cone can be changed resulting in varying levels of clarity of the mirrored reflection. The aperture is based on the roughness of the PBR material texture. This means that this BRDF allows a more reflective material to have a smaller cone which creates clearer mirror-like reflections. A rougher material will instead scatter the light more diffusely,

simulated with a wider cone leading to a more approximated indirect specular lighting sample of the surroundings. [1]

The cones in voxel cone tracing will be assembled in a similar way to Figure 17.

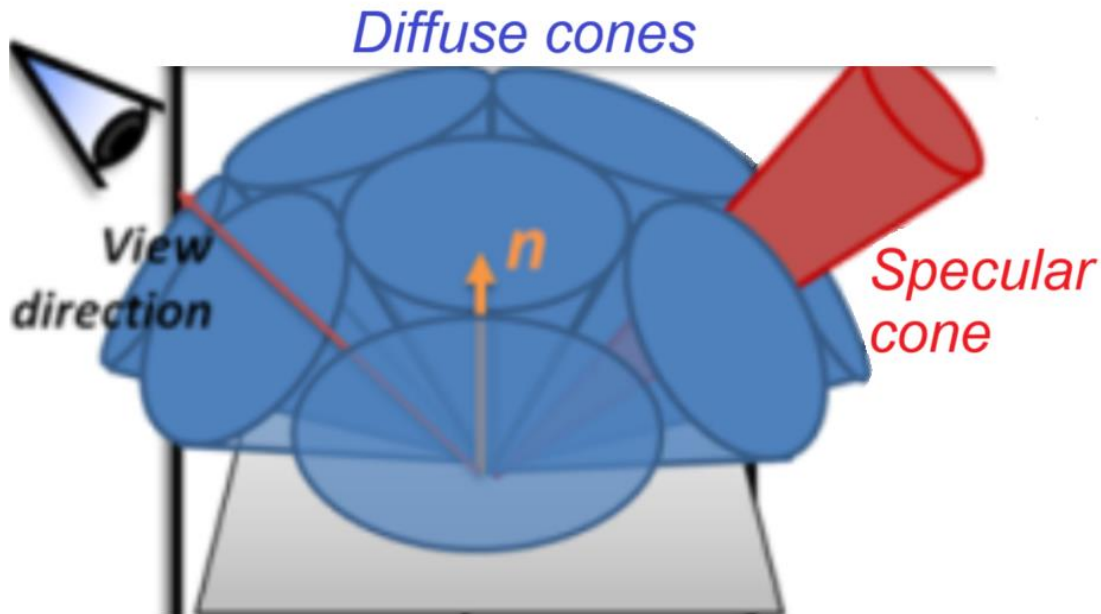


Figure 17: multiple diffuse cones over Ω^+ and one smaller specular cone sampling indirect diffuse and specular lighting respectively. [1]

The traced cones are approximating, through voxel sampling, the indirect light from a bigger area rather than infinitesimal rays that sample from specific singular surface points as used in path tracing. The main appeal of cone tracing is that the cones can with this approximation create the illusion that many rays have been cast when only a few cones have been used. Normally when using a path tracer after rays have been cast a *denoiser* has to be used. A denoiser is a technique used to decrease grainy spots and discoloration in images. Denoisers are needed after using a path tracer since the result will be grainy and uneven and the denoiser can smooth the result out [28]. But now the effects of denoising or doing multiple ray traces is gained for free by using cone tracing. The sampling results of the cones are already an averaged sample giving accurate indirect light results because of the technique [1].

By stepping along and “tracing” the cone the light data is sampled from the voxel data for each cone trace step and so all lighting information within the cone is accumulated. What makes this method so efficient is that the flexible LOD provided by the mipmap of the voxels is calculated and scales with the diameter of the cone base. Meaning, even though more voxels fit as the cone becomes bigger, instead of sampling many small voxels at a bigger cone diameter, a lower mipmap level is used to sample fewer bigger voxels which are already averaged [1], see Figure 18.

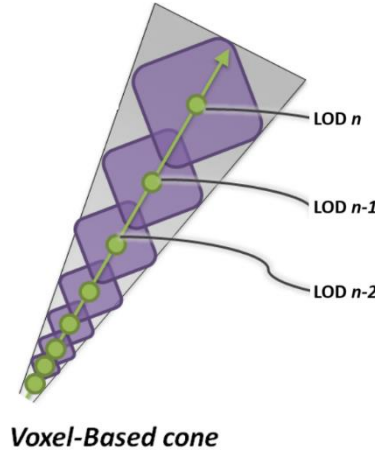


Figure 18: *voxel LOD depends on the cone's base diameter at the current sampling step.*

The implication of this is that many smaller voxels are combined into one bigger voxel through mipmapping with a single averaged data that can be sampled. Voxel cone tracing becomes extremely efficient because of this, since the filtered voxel information allows the cone-based sampling to rapidly approximate the indirect light contribution from a large amount of geometry [1][16].

The usage of a GPU supported trilinear filtering allows for smooth sampling between mipmap levels when the cone diameter results in a LOD awkwardly in between two mipmap levels. Then the trilinear filtering will sample from the two relevant LOD and then interpolate between them according to the fractional part of the LOD value. This way the cones will continue to step through the scene and sample lighting at each step until either of two criteria are met. A max stepping distance is reached, or a significant accumulated occlusion value is reached. Where the occlusion a is the total light blockage so far from the solid objects in the cone. The total accumulated indirect light and a is updated per step using the classical volumetric front-to-back accumulation [1], see Eq6 and Eq7:

$$c += (1 - a) * a_2 * c_2$$

Eq6: Indirect light accumulation per sampling step

$$a += (1 - a) * a_2$$

Eq7: Occlusion accumulation per sampling step

Here, c is the color sampled so far representing the reflected light towards the cone origin and a is the sampled occlusion so far. While a_2 and c_2 is the occlusion and color sampled in the voxel at the current step. The multiplication $a_2 * c_2$ represents that the total color of the voxel that is averaged by the content inside the voxel where solid occlusive objects contribute with colors $(R, G, B) \geq (0,0,0)$ and empty space containing no color dilutes with $(R, G, B) = (0,0,0)$ [1].

Front-to-back compositing is a method that processes and blends the voxel data starting nearest the cone tip starting position and progresses towards the base of the cone. It means that objects

closer to the point of interest p are the ones occluding the objects in the back, that are not visible from p point of view. In practice this means that if a cone hits an edge of something but is not stopped completely then the sampling of objects afterwards will have a smaller effect on the result to simulate that parts of the cone have been blocked [1].

Ambient occlusion, or AO for a point p , affects and has a dampening effect on the indirect lighting. It provides soft shadows especially at sharp concave spaces and creases such as room edges, corners but also just surfaces that have many objects close to it. This is because the indirect light has a harder time reaching these concave places. In corners for example the light tends to bounce back into the scene if it's not precisely around half of the corner angle. In a sense the AO measures how accessible each point is to the environment's indirect light. To be formally correct AO looks a lot like the rendering equation and is an integral over the continuous hemisphere but with the visibility function $V(p, \omega)$ [7], see Eq8:

$$A(p) = \frac{1}{\pi} \int_{\Omega} V(p, \omega) (\cos \omega) d\omega$$

Eq8: Formula for the AO at point p .

Like with the indirect diffuse lighting the hemisphere Ω^+ is approximated by partitioning and summarizing a from each cone and applying cone weights. This means that AO can be calculated at the same time, with the same cones, in a similar manner as when the indirect diffuse lighting is calculated in the fragment shader.

The visibility function determines if a ray intersects the scene and thus leads to occlusion. It is substituted to a weight function $f(\text{cone diameter})$ that decays with distance. This is done since indoor scenes with little to no visible sky, such as the Sponza scene, will almost always have ray intersections. That's why $f(\text{cone diameter})$ instead decays with distance meaning that the occlusion of solid objects at a sample step are less occlusive if located far away (where the cone has a bigger cone diameter) [1].

If λ is a parameter that controls the rate of decay with distance and d is the cone diameter, then $f(\text{cone diameter})$ is calculated with the following equation Eq9 [1]:

$$f(d) = \frac{1}{1 + \lambda * d}$$

Eq9: Weight function $f(d)$ which decays as the cone base diameter grows.

3.3.4 Multiple light bounces

One unique and useful aspect of voxel cone tracing is how easy it is to implement multiple indirect light bounces and be able to render it at real time speeds. This is due to the nature of voxels and how this method injects light into them. For a singular bounce only, the direct light is injected into the voxel and then the standard voxel cone tracing algorithm is run on to get the direct and indirect diffuse light together with specular light. However, it is possible to instead of using a fragment shader to render the result of one bounce to the screen to instead use a compute shader for an additional bounce. Afterwards each voxel in the scene will calculate and save indirect diffuse light using cone tracing and save the result in another sparse 3D texture.

This way another texture with both direct light and an additional light bounce is created. It is worth noting that only indirect specular light can be calculated using this method as specular light requires a camera with a viewing direction to calculate [1].

Using this method does require additional memory consumption for two things. Firstly, an additional sparse 3D texture is required to save the result. This method also requires a third texture to be used, a sparse 3D texture for storing the normal of every voxel so that the compute shader can perform accurate calculations. Therefore, the memory consumption is tripled for creating an additional bounce. However, for more bounces than 2 the memory consumption does not increase further. This is because the original sparse texture used to save direct light can be reused to store a further additional bounce. The textures can be swapped back and forth to store an unlimited number of bounces with memory consumption in mind, but the time required does of course increase per bounce calculated [1].

What is done here in practice is applying the solution to the rendering equation problem *b*) discussed in Ch 3.2.2. That is setting a fixed number of bounces and then setting the order of the calculation, calculating one bounce at a time for each voxel using *Eq4*. Then store the latest indirect light information each bounce iteration. The result is that light simulations can get more realistic global illumination at the cost of more cone tracing passes [1][7].

4 Method

4.1 Implementation

4.1.1 Setup and Hardware

One of the main goals of this report was to gain an understanding of how different voxel texture sizes and different light behavior affect the performance, visual result and memory consumption of Voxel Cone Tracing. Therefore, it was decided to implement a voxel cone tracer that can use different texture sizes and both 1 and 2 light bounces. This thesis uses the programming language C++ with OpenGL as the GPU API. C++ was used because of its high speed and versatility. Since it is one of the standard languages used in the gaming industry and a highly popular language for performance demanding programs it was deemed as a suitable language for this purpose. OpenGL was used because of its widespread availability and ease of use. At first the Vulkan API was considered, to be able to access the RTX cores available in the RTX 3060 laptop. Because of the time limitations of this project and the lower-level nature of the Vulkan API this was decided against. The scene that was chosen for testing and light simulation was the “Sponza” scene created by Crytek adapted into a GLTF 2.0 file format by KhronosGroup, the creators of OpenGL [17].

All implementations and results were done using an RTX 3060 laptop with an AMD Ryzen 5900HS processor.

4.1.2 Voxelization of the scene

To voxelize a scene there are several steps to be taken. The steps taken are a practical implementation of the steps explained in chapter 3.3.1.

The first step is to use the vertex shader to transform the vertex positions from a mesh into the voxel grid coordinate system. In practice this means that the scene is transformed so the coordinates fit into the sparse 3D texture directly. It is important to disable depth testing before this step as the entire scene needs to be voxelized and not just the visible textures. Afterwards the geometry shader is used to project every primitive on the same 2D surface for later.

The second step is during rasterization. The only things that need to be done at this stage from the users’ end is to change the viewport resolution into the same resolution as the voxel texture size. A voxel texture size of 512^3 needs a viewport resolution of 512×512 at the time of voxelizing. This is because every pixel on the screen at every depth level will be transformed into voxels. If there is a higher resolution there will be an overlap in voxels and if there is a lower resolution, then there will be missing voxels. The other thing is that conservative rasterization needs to be enabled to remove cracks in the voxelization. This step is done behind the scenes with all the work being done in the first and final step, but it is still important.

The final step is in the fragment shader where the voxels are saved to the texture. In the fragment shader a shadow map is used to calculate the direct light that is saved in each voxel. This is

done by simply calculating the direct light of the scene normally and then saving the light result at the voxels location. The result is that every mesh is saved to the texture, although many of them will be completely black as no light reaches them. Completely black voxels are still needed as they block light from traversing through them when cone tracing through the texture later. The opacity value of each voxel is saved, see figure 19 for further implementation details.

Algorithm 1 Simplified Geometry and Fragment Shaders for Scene Voxelization

```

Geometry Shader:
1: vec3 p1  $\leftarrow gl\_in[1].gl\_Position.xyz - gl\_in[0].gl\_Position.xyz$ 
2: vec3 p2  $\leftarrow gl\_in[2].gl\_Position.xyz - gl\_in[0].gl\_Position.xyz$ 
3: normal  $\leftarrow \text{normalize}(\text{cross}(p1, p2))$ 
4: float nDotX  $\leftarrow \text{abs}(\text{normal}.x)$ 
5: float nDotY  $\leftarrow \text{abs}(\text{normal}.y)$ 
6: float nDotZ  $\leftarrow \text{abs}(\text{normal}.z)$ 
7: axis  $\leftarrow (nDotX \geq nDotY \wedge nDotX \geq nDotZ)?1 : (nDotY \geq nDotX \wedge nDotY \geq nDotZ)?2 : 3$ 
8: mat4 projectionMatrix  $\leftarrow \text{axis} == 1?ProjX : \text{axis} == 2?ProjY : ProjZ$ 
9: for  $i = 0$  to  $gl\_in.length()$  do
10:   UV  $\leftarrow \text{vertices}[i].UV$ 
11:   position_depth  $\leftarrow \text{vertices}[i].position\_depth$ 
12:   gl_Position  $\leftarrow \text{projectionMatrix} \times gl\_in[i].gl\_Position$ 
13:   EmitVertex()
14: end for

Fragment Shader:
15: vec4 materialColor  $\leftarrow \text{texture}(\text{diffuse}, UV)$ 
16: float visibility  $\leftarrow \text{texture}(\text{ShadowMap}, \text{vec3}(\text{position\_depth}.xy,$ 
17:    $(\text{position\_depth}.z - 0.001)/\text{position\_depth}.w))$ 
18: ivec3 camPos  $\leftarrow \text{ivec3}(gl\_FragCoord.x, gl\_FragCoord.y, \text{VoxelDimensions} * gl\_FragCoord.z)$ 
19: if axis == 1 then
20:   texPos.x  $\leftarrow \text{VoxelDimensions} - \text{camPos}.z$ 
21:   texPos.z  $\leftarrow \text{camPos}.x$ 
22:   texPos.y  $\leftarrow \text{camPos}.y$ 
23: else if axis == 2 then
24:   texPos.z  $\leftarrow \text{camPos}.y$ 
25:   texPos.y  $\leftarrow \text{VoxelDimensions} - \text{camPos}.z$ 
26:   texPos.x  $\leftarrow \text{camPos}.x$ 
27: else
28:   texPos  $\leftarrow \text{camPos}$ 
29: end if
30: texPos.z  $\leftarrow \text{VoxelDimensions} - \text{texPos}.z - 1$ 
31: vec4 metallicRoughnessColor  $\leftarrow \text{texture}(\text{metallicRoughnessTexture}, UV)$ 
32: float roughness  $\leftarrow \text{metallicRoughnessColor}.g$ 
33: imageStore(VoxelTexture, texPos, vec4(materialColor.rgb * visibility, 1.0))
34: imageStore(VoxelNormalTexture, texPos, vec4(normal.x, normal.y, normal.z, roughness));

```

Figure 19: Code for voxelizing a mesh

Figure 19 shows the method for voxelizing a mesh using a geometry shader and fragment shader. This implementation stores the normal of each voxel so that it can be used later in the compute shader. In a compute shader no normal from the mesh is available therefore the compute shader needs the normal information stored in another sparse 3D texture. Figure 20 shows the result of the voxelized scene using a 512³-voxel texture size with direct light saved. These voxels are later used to calculate the first light bounce.



Figure 20: Initial voxelization with direct lighting using 512^3 texture size

4.1.3 Voxel Cone Tracing implementation

The voxel cone tracing algorithm calculates light by accumulating several types of light and adding them together to simulate realistic light as described in section 3.3.3. Below is Figure 21 which is the algorithm used to trace cones and sample voxels for diffuse radiance as well as specular radiance.

Algorithm 1 Voxel Cone Tracing LOD sampling

Data: cone aperture: θ , origin: $start$, direction: dir
Result: captured radiance of cone

- 1: $radiance \leftarrow \text{vec3}(0)$
- 2: $distance \leftarrow \text{VoxelSize}$
- 3: $occlusion \leftarrow \text{vec3}(0)$
- 4: $pos \leftarrow \text{vec3}(0)$
- 5: $origin \leftarrow pos + Offset \times dir$
- 6: **while** $occlusion < 1$ **do**
- 7: $pos \leftarrow start + dir \times distance$
- 8: $distance \leftarrow distance + 2 \times \tan(\theta/2) \times distance$
- 9: $LOD\ level \leftarrow \log_2(distance)$
- 10: $radiance \leftarrow radiance + \text{sampleVoxels}(distance, LOD\ level) \times (1 - occlusion)$
- 11: $occlusion \leftarrow occlusion + (1 - occlusion) \times radiance\ occlusion$
- 12: **end while**
- 13: **return** $radiance$

Figure 21: Cone tracing algorithm

4.1.4 Final color output

To calculate the final fragment color output, Eq5 from earlier is used:

$$Final\ Color = DiffuseRadiance + SpecularRadiance$$

Where each term will be calculated in the following way based on the equations in chapter 3:

Diffuse radiance

$$diffuseRadiance = AO * indirect\ DiffuseLight + direct\ DiffuseLight$$

Where the *Ambient Occlusion (AO)* and *indirect DiffuseLight* are both calculated by sampling through the scene by cone tracing with 6 cones. The cone apertures, angles and weight can be seen in the appendix (CA) 1. As mentioned in Ch 3.3.3 the indirect diffuse light can be calculated by sending several cones out from the surface to cone trace the scene. The sum of all cones will then be added together for the result. The indirect diffuse light calculation can be written as Figure 22 shows.

Algorithm 1 indirect diffuse light and ambient occlusion

```
1: function INDIRECTLIGHT( )
2:   vec4 color ← vec4(0)
3:   ambient_occlusion ← 0.0
4:   for i ← 0 to NUM_CONES do
5:     color ← color + coneWeights[i] * coneTrace(angle, position, normal
      * coneDirections[i])
6:     ambient_occlusion ← ambient_occlusion + coneWeights[i] * ambi-
      ent_occlusion
7:   end for
8:   ambient_occlusion ← 1.0 - ambient_occlusion
9:   return color * ambient_occlusion
10: end function
```

Figure 22: calculating indirect diffuse light and ambient occlusion

Here the function traces every cone based on the normal surface. Every cone is angled so that as much of the hemisphere Ω is covered. The function calls another function, *coneTrace* which is the algorithm that Figure 21 showcases. Ambient occlusion also gets returned by the cone trace algorithm indirectly and is used to calculate the color to be returned. The main purpose of ambient occlusion is to darken areas such as corners and concavities which should better simulate how real light behaves as described in chapter 3.3 [4].

The term *directDiffuseLight* is calculated with the Lambertian model together with a shadow map deciding its visibility. The shadow map simply renders the scene from the light source point of view. Then a depth test can compare the fragments to be rendered with the

camera point of view and the shadow map. This way the visible fragments in the shadow map will be subject to direct light [1].

$$\text{Direct Diffuse Light} = \text{visibility} * \max(0, \text{dot}(\text{normal}, \text{lightDirection}))$$

Specular radiance

Specular radiance is dependent on where the camera is and how light would bounce on the surface to hit the camera. The equation can therefore only be done in real time and not precomputed and stored in a voxel texture.

$$\text{Reflect Direction} = \text{normalize}(I - 2.0 * \text{dot}(N, I) * N)$$

where I is the incoming camera direction and N is the surface normal. The result is a vector which points in the direction of the reflected direction of the surface.

Afterwards cone tracing is used to receive the final specular light. Since this implementation uses PBR, the roughness of the surface is known and therefore the cones diameter is based on it. The rougher the texture the bigger the diameter. This aims to represent that a rougher more uneven surface spreads light more than a reflective and smooth surface. The cone is sent out in the reflected direction from the surface normal with a diameter of close to 1° of the hemisphere while the surface is completely smooth. This diameter increases with the roughness of the surface. It increases up to a maximum of 180° coverage of the hemisphere when a surface is completely rough to emulate the effect discussed in chapter 3.3.3.

$$\text{Indirect Specular Radiance} = \text{cone trace}(\text{cone Width}, \text{surfacePosition}, \text{Reflect Direction})$$

Where *cone trace* is the function implemented based on Figure 21.

Direct specular radiance does not use cone tracing and is calculated using an equation similar to direct diffuse radiance equation:

$$\text{direct specular radiance} = \text{visibility} * \max(0, \text{dot}(\text{reflectDir}, \text{light direction}))$$

This equation uses, as discussed earlier in chapter 3.2.6, a geometry term that depends on the camera's point of view that hits a surface and the light's direction. Also, as previously covered in chapter 3.3.3, the PBR is also used here to scale the shininess of the specular reflection.

4.2 Evaluation methods

The results are evaluated using several metrics which aim to evaluate performance, visual quality and accuracy. Performance is evaluated by timing the overall program as well as each main function the program does such as rendering, light calculation, voxelization of scene etc. The timings are done for each different voxel texture size used with both one bounce and two

bounces compared. Comparisons between static scenes and dynamic scenes performance are also made between the variations in bounces and texture size. Visual quality will only be assessed by humans as several aspects of the quality are relative and the focus of the thesis is performance. That is, if the visual results are somewhat realistic without massive issues, the visual results will be deemed good enough for usage in something like a video game. Reference images from Blender, an offline industry standard render application will be provided to compare visual results. Blender uses accurate offline ray tracing resulting in zero visual artifacts and can thus be used as sort of a ground truth when identifying artifacts from the voxel cone tracing implementation [9].

5 Results

All performance metrics are measured using Nvidia Nsight Graphics [8]. A reference picture has been created using Blender to get a more accurate path traced result to which the result can be compared to.

5.1 Performance Results

Performance has been measured for three different voxel texture sizes, 128^3 , 256^3 and 512^3 . Each individual step of the algorithm was measured individually using Nvidia Nsight Graphics. This includes the time to create a shadow map, voxelize the scene, mipmapping the textures, voxel cone tracing an additional light bounce and finally voxel cone tracing the final scene. Every render is done with a resolution of 1920x1080p and rendering the Crytek Sponza scene using PBR textures obtained from Khronos Group [17].

The first table shows the frame rate of the different voxel texture sizes while the scene is dynamic compared to when the scene is static. A static scene means that no voxel textures are updated, and the program is only rendering the scene normally and only calculating a single light bounce in real time. A dynamic scene means that new voxels are created each frame and if there are 2 bounces there is a compute shader which will calculate an additional light bounce before the actual rendering of the scene. For both a dynamic scene and a static scene the scene is always cone traced at least once, but there are many more operations required before the final bounce is calculated in a dynamic scene.

Performance of Sponza scene – 1080p – Dynamic vs Static		
Voxel Resolution	Dynamic Scene (FPS)	Static Scene (FPS)
Voxel Cone Tracing 2 Bounces, 512^3	4.3	120.2
Voxel Cone Tracing 1 Bounce 512^3	24	120.2
Voxel Cone Tracing 2 Bounces, 256^3	29	119.92
Voxel Cone Tracing 1 Bounce, 256^3	88	119.92
Voxel Cone Tracing 2 Bounces, 128^3	104	120.1
Voxel Cone Tracing 1 Bounce, 128^3	119	120.1

Table 1: Showing the average frame rate of the Voxel Cone Tracing algorithm for a static scene and for a dynamic scene in 1920x1080p over different texture sizes and light bounce amounts.

Performance of Sponza scene – 1080p – Time Taken for Operations									
Voxel Re- solution	Shadow (ms)	Voxelize (ms)	Mipmap (ms)	Additional Bounce (ms)	Mipmap (ms)	Draw & Cone trace (ms)	Total time (ms)	FPS	
2 Bounces, 512 ³	0.08	0.47	33.2	155.77	33.2	7.09	229.81	4.35	
1 Bounce 512 ³	0.08	0.47	33.2	-	-	7.09	40.84	24.4	
2 Bounces, 256 ³	0.08	0.36	4.3	19.82	4.3	6.29	35.15	28.44	
1 Bounce, 256 ³	0.08	0.36	4.3	-	-	6.29	11.03	90.66	
2 Bounces, 128 ³	0.08	0.33	0.78	3.83	0.78	5.05	10.85	92,16	
1 Bounce, 128 ³	0.08	0.33	0.78	-	-	5.05	6.24	160	

Table 2: Showing a detailed breakdown of the time each operation takes in Voxel Cone Tracing using Nvidia Nsight.

5.2 Visual Results

Figures 22-26 showcase different camera and direct light angles of the Sponza scene rendered with different amounts of light bounces and voxel texture sizes. They all illustrate the indirect lighting contribution. Angles in Figure 22 also showcase smoothened shadow edges from the open radius windows direct lighting. Angles in Figure 25 showcase the color bleed phenomena from the curtains and light leakage artifacts. Lastly, Figure 26 showcases different indirect light approximation errors around concavities and creases. To illustrate a demonstration of visual artifacts the indirect lighting was scaled with a high coefficient of 10 for Figure 26.

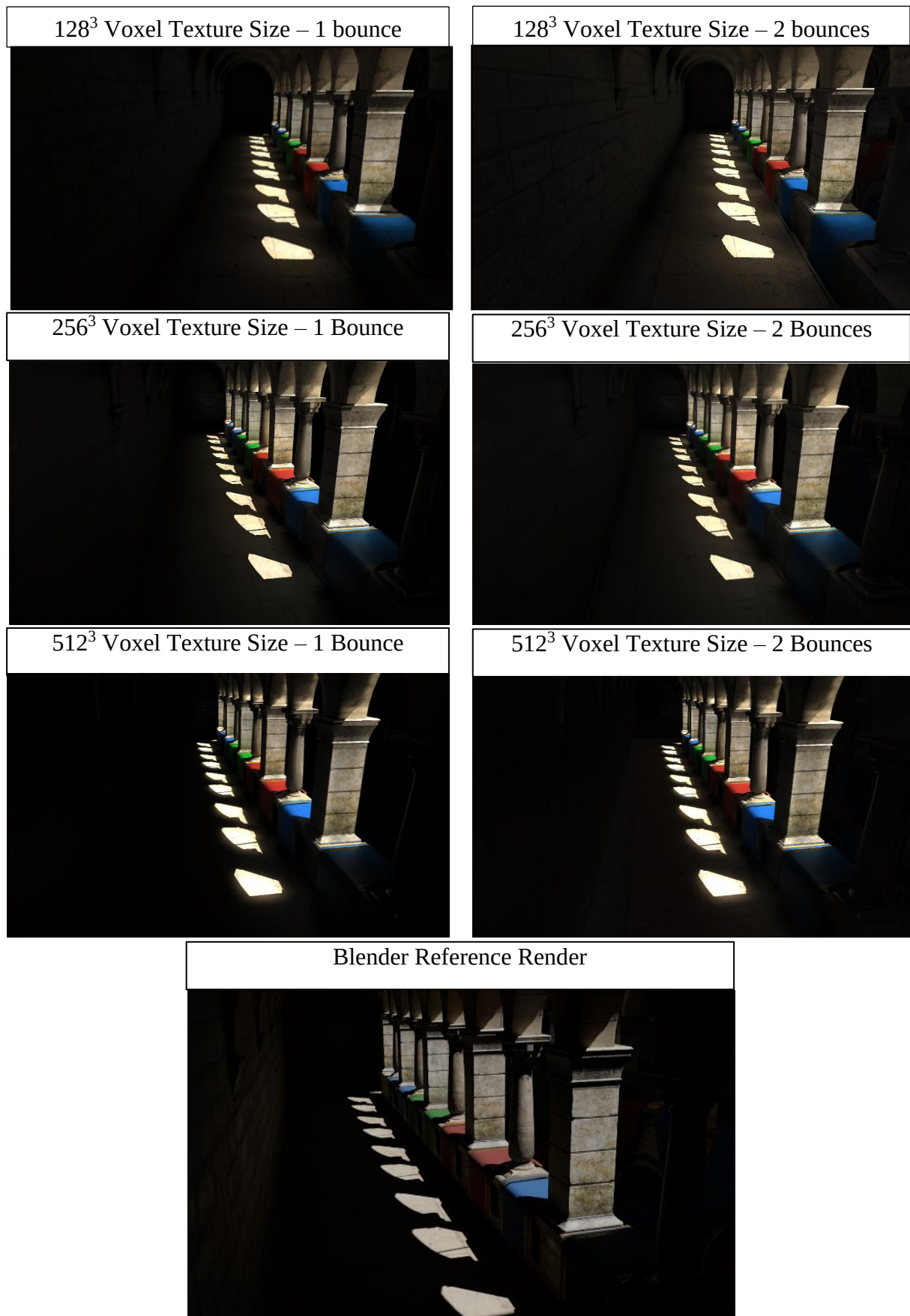


Figure 23: Visual comparison between different voxel texture sized and light bounces

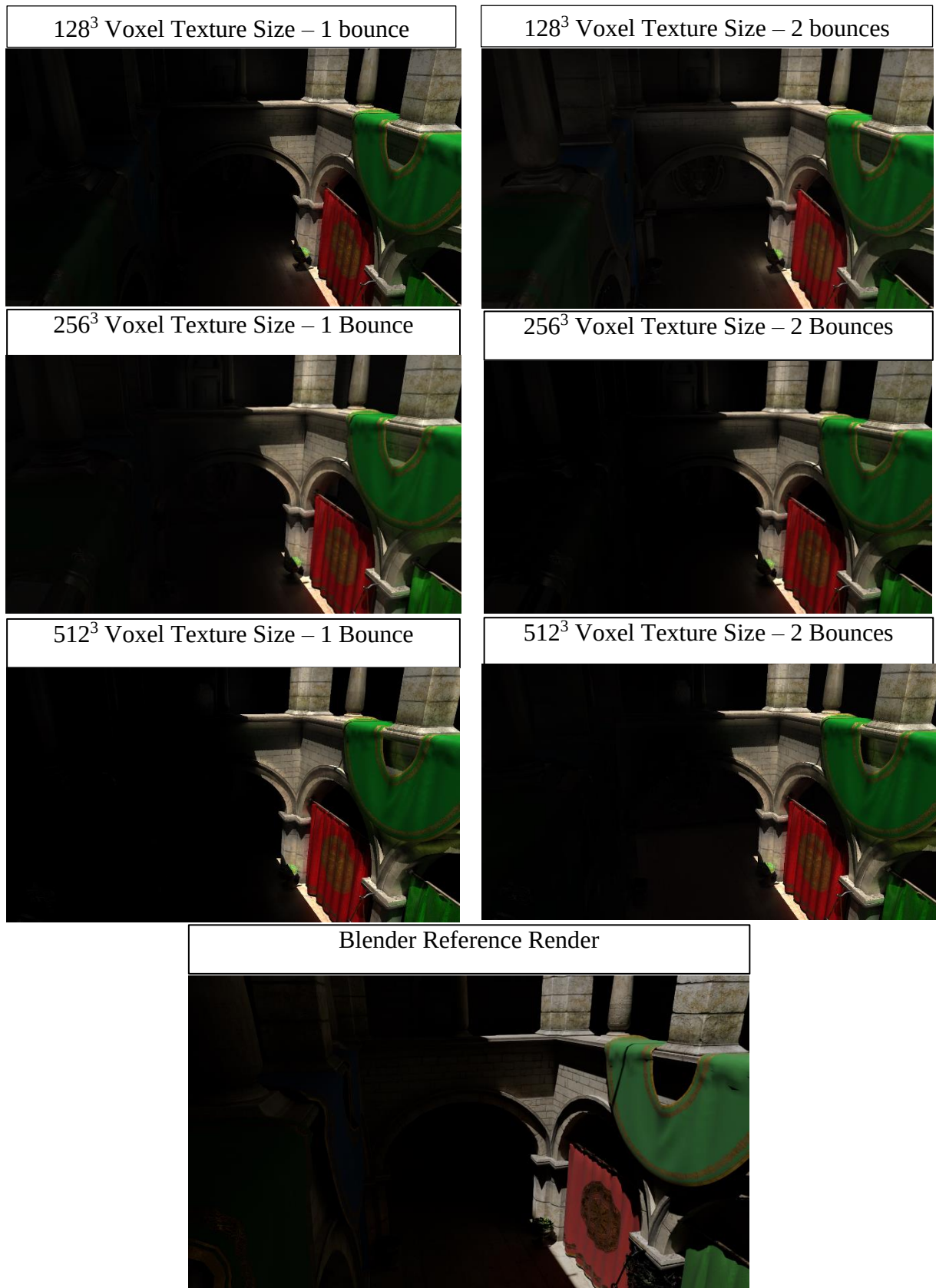


Figure 24: Visual comparison between different voxel texture sized and light bounces

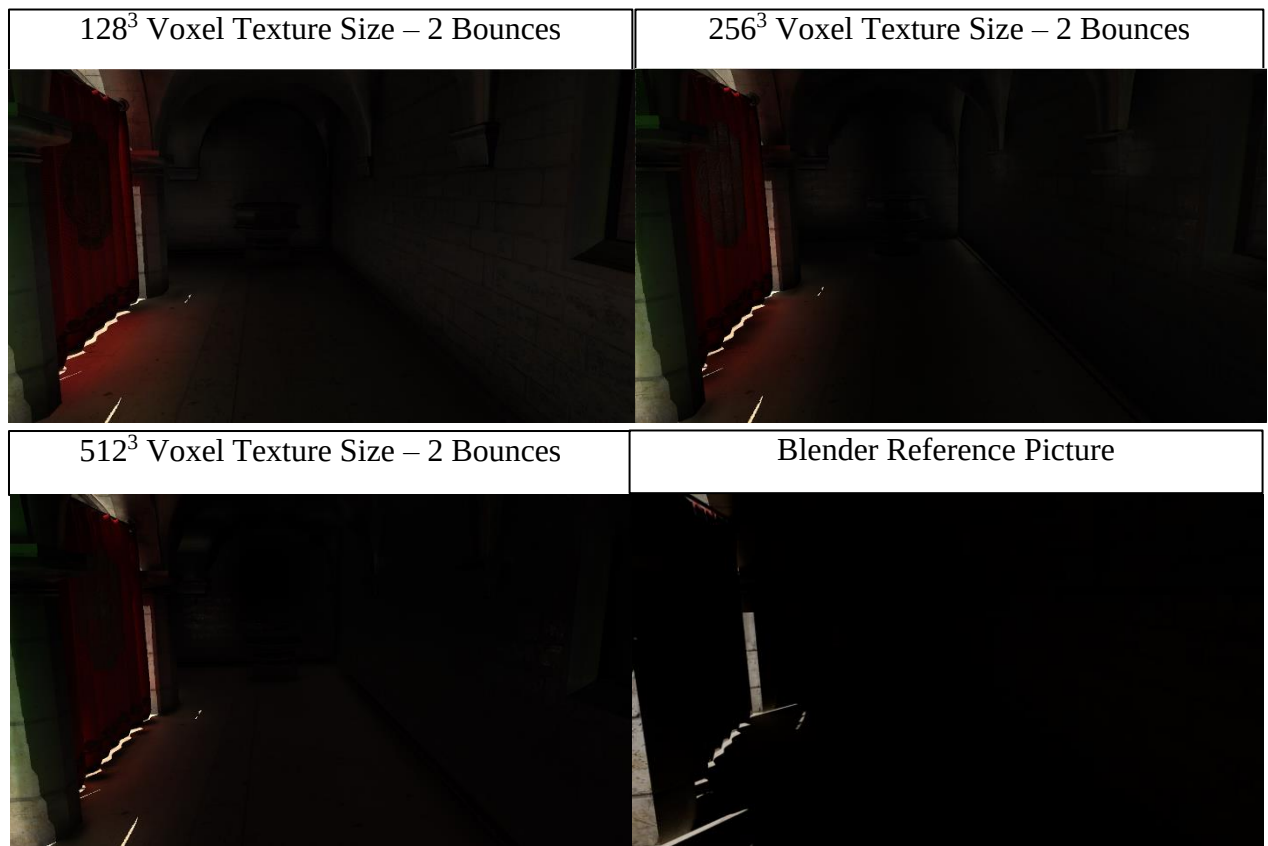


Figure 25: *Showcasing light leaks at different texture sizes*

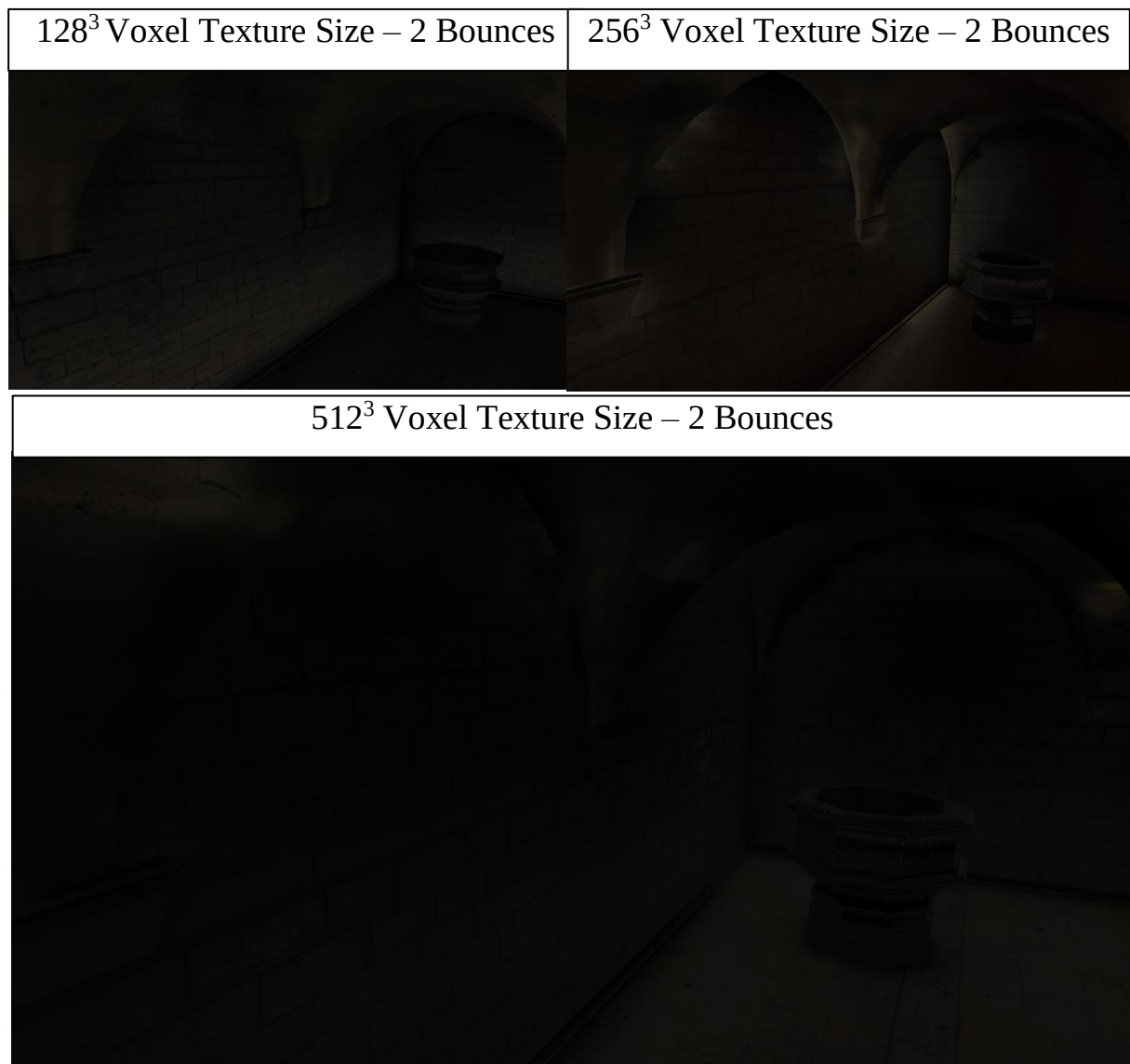


Figure 26: *Showcasing the light emission error from corners when there is no direct light source*

All the results show that the three different voxel texture sizes all achieve global illumination, with more realistic and prevalent indirect lighting with multiple bounces although with an increase in visual artifacts. Even though artifacts are present the visual quality otherwise is comparable with Blender in many aspects and still provide global illumination and in real time at that.

5.3 Memory Consumption of Voxel Textures

Memory consumption - Sponza Scene				
Voxel resolution	Dense texture size (MB)	Sparse texture size (MB)	Total chunks of texture allocated	Allocated %
128 ³	8	6	48	75
256 ³	67	25	192	37.5
512 ³	512	146	1120	27.3

Table 3: *Memory consumption of the textures used to store the voxels. The dense textures are normal 3D textures, and the sparse textures are sparse 3D textures using the GL_ARB_sparse_texture extension in OpenGL*

6 Conclusion and Future work

6.1 Discussion

Although voxel cone tracing is an old technique that is not used much by the video game industry anymore, mostly because of the memory issues discussed earlier, it is still a technique that renders mostly realistic lighting with high performance. It is also a technique which is appropriate for complex scenes with dynamic lighting.

With modern graphics cards having a bigger VRAM capacity and many modern games wanting to utilize ray tracing there is a possibility that voxel cone tracing might see more mainstream attention in the future. Especially in video games where the scenes are somewhat small with a lot of complexity and/or scene variance. Perhaps in the future the inherent memory consumption issue with Voxel Cone Tracing will be completely solved by hardware advancements. But at the current time many video games will have scenes which are too big to use Voxel Cone Tracing effectively without some compromises with current hardware.

The results show that voxel cone tracing can create realistic lighting in images with real time speeds for one light bounce and almost real time speeds for two light bounces, although with some visual artifacts and a high memory consumption. The most unexpected result was the time taken to mipmap the voxel textures. As mentioned earlier in Ch 3.3.2, Nvidia does not have a built-in method for mipmapping a 3D texture on the GPU. Mipmapping therefore had to be implemented from scratch to run in a compute shader. This result is probably a consequence of a poor implementation of mipmapping 3D textures as it should theoretically be much faster.

Another somewhat unexpected result is the slow speed of the second light bounce calculation. Calculating the bounce using the fragment shader and rendering the scene took only a fraction of the time that the compute shader took to calculate one light bounce. A big part of the difference in computational time is probably that the fragment shader does one cone trace per triangle in the camera's field of view. This contrasts with the compute shader which cone traces every single voxel in the scene to make sure that every voxel is as accurate as possible. This means that the compute shader workload scales linear with the size of the voxel texture while the fragment shader which does the final bounce scales with the complexity of the scene. The fragment shader computation scales with the number of triangles in the scene and the camera's field of view while the compute shader scales with the voxel texture size. The implication of this is that the time required for Voxel Cone Tracing in the fragment shader is mostly dependent on the complexity of the scene. This contrasts with the Voxel Cone Tracing done in the compute shader which is mostly reliant on how large the voxel texture size is. For comparison the Sponza scene has around 260 000 triangles while a voxel texture of size 512^3 has around 134 000 000 voxels. Which means that the compute shader has a maximum of around 516 times as many operations as the fragment shader.

6.1.1 How do different voxel sizes and light behavior affect performance?

Table 2 in the results show that the voxel texture size heavily affects the performance in some key areas, most notably during mipmapping and saving the additional bounce to the voxel texture using a compute shader. This is an expected result as the computational complexity increases linearly as the texture sizes increase. To create an additional bounce using the compute shader with 256^3 voxel texture size the number of calls to the compute shader is 16 777 216. Compare that to a texture size of 512^3 which must do 134 217 728 calculations, an increase of 8 times as many calls. It is important to note that many of the calls to the compute shader result in early exits as most of the calls will end up in texture spots where there are no voxels saved and thus no calculations needed.

One solution to the problem discussed is to only call the compute shader for only at spots where the sparse voxel texture is known to be allocated to reduce redundant GPU work. An implementation of this was initially tested but due to time constraints this was decided against as some smart solutions are needed. For example, calling for maximum GPU efficiency, few compute shaders with a lot of work is wanted instead of many small work groups. This means that for a workable implementation there needs to be a system to determine where there are clusters of voxels in the texture and call the compute shader to calculate a big area instead of individual voxels.

Overall, the performance speed of the actual Voxel Cone Tracing was incredibly fast for a single bounce. Creating the highest level of detail voxels and then calculating a single bounce took only a few milliseconds as table 2 shows. As mentioned earlier the performance issues were mainly the mipmapping of textures at higher resolutions as well as the calculation of a second light bounce at higher texture sizes. It was expected that these operations would take a big portion of the rendering time, but the time taken for larger texture sizes shows that the implementation has some problems. Calculating a second bounce in a compute shader should not take around 50 times longer than calculating the second bounce while using a larger texture in a fragment shader. In conclusion, the performance of Voxel Cone Tracing with only a single light bounce is fast at every texture size, but more bounces do not scale as well with an increase in texture size.

6.1.2 How do different voxel sizes and light behavior affect memory consumption?

Memory consumption is one of the biggest issues with Voxel Cone Tracing and the results show this quite clearly. As table 3 shows, using a sparse texture with a voxel texture size of 256^3 consumes 25 MB compared to a voxel texture size of 512^3 consumes 146 MB. This might not seem like much, but it comes with a lot of computational cost to update the textures. For many video games and other real-life usages, the scenes are many times larger than the Sponza scene used in this thesis. This means that for many other usages it will not be enough with the smaller textures as the voxels will be more spread out to cover a bigger area. The consequence of this is a requirement for more voxels which means that the memory consumption can increase heavily.

While calculating multiple light bounces the memory consumption triples as mentioned in chapter 3.3.4. The reason for this is the need for three sparse 3D textures instead of just one. Memory consumption is therefore a bigger problem for implementations of Voxel Cone Tracing with multiple bounces. This is probably one of the reasons for poorer performance with multiple light bounces. While using larger texture sizes, more memory needs to be read and written to each frame which can often be a big bottleneck. The increased memory consumption is probably also one of the reasons why almost no implementations of Voxel Cone Tracing use multiple light bounces. It is worth noting that the memory requirement will always be three times the normal usage as brought up in chapter 3.3.4. The implications of this are that if the memory consumption of Voxel Cone Tracing with a single bounce is trivial in the future, then the extra consumption of multiple bounces would most likely also be trivial.

6.1.3 How do different voxel texture sizes and light behavior affect image quality?

By comparing the side-by-side pictures with different light bounces the observation can be made that more bounces increase the overall lighting since the indirect lighting contribution has been increased. This was to be expected since the second light bounce is calculated using information the first bounce created as in *Eq4*. Furthermore, by comparing the voxel texture sizes it can be observed that lower voxel texture resolutions tend to lead to too strong ambient indirect lighting, i.e. a lighter scene. A voxel texture size of 512^3 instead still provides indirect lighting but overall gives a more realistic amount of indirect light even if a bit dimmer. This is to be expected as smaller voxels more accurately represent the scene. For example, the color bleeding because of light leaking from the thin curtain objects in figure 25 with a texture size of 512^3 is much less than in the other renders with a lower voxel texture size. This is because the curtain is more accurately represented by the smaller sized voxels. It was clear that when comparing the Voxel Cone Tracing renders to the Blender renders that the Voxel Cone Tracing render has too much light leaking through. Too much light leakage is not a detrimental effect as it can often look quite realistic, but it is still an error.

Both the frequency, exact locations and amount of miscolor of the visual artifacts change greatly with the voxel texture size and number of bounces. The gravity of the discolored visual artifacts increases with two bounces, comparing the images in figure 26. This is because when calculating the second bounce by cone tracing, the sampled values are already built on cone trace approximation. Since sampling directions are always fixed and not random, it means that for each bounce the artifact will continue to sample from the same inaccurate voxel. This sampled inaccurate voxel could for example be a victim of light leakage due to isotropic faces. The implementation created only implemented one or two light bounces but could easily be increased to more, although at a cost of performance. The one thing to note which was mentioned earlier is the increased risk for light leaking and artifacts from multiple bounces

The Voxel Cone Tracing implementation has created global illumination as is shown by the rendered images in the results. Side effects are shown such as smooth shadows, color bleeding, indirect and direct lighting. All three texture sizes hold up quite well compared to Blender visually but comparing them directly reveals light artifacts that lowers the image quality. It can also be noted how the artifact locations change between different voxel texture sizes. This is probably because of different LODs providing different amounts of inaccurate voxels.

However, true for all texture sizes is that artifacts almost exclusively appear on concavities and creases. Ambient occlusion helps reduce this but does not seem to solve this issue entirely. Perhaps with some further tweaking of parameters this issue could be resolved better in the future.

6.2 Further improvements

There are many areas of improvement for the current implementation of Voxel Cone Tracing in this thesis. The key issues as outlined earlier are light leaks, memory consumption and performance handling large 3D textures. With more optimizations the program could probably reach real time dynamic light with playable frame rates at higher voxel counts with more bounces. Another big improvement would be to only update the voxels when needed, that is if there is no possibility of a voxel being changed then there is no need to calculate anything for that voxel. Finally, a possible big improvement to the compute shader where an additional bounce is calculated would be to only do calculations on areas which are dense with voxels instead of sending the compute shader to the entire 3D texture and checking if there is a voxel there. This would theoretically increase the speed of the compute shader by around 4 times for 512^3 textures as the result of table 2 shows that about 27% of the texture is dense with voxels. Hopefully this method can be returned to later to implement these improvements and compare the results to further quantify how good different methodologies are.

6.3 Conclusion

Voxel Cone Tracing has recently started gaining traction again utilizing recent progressions in modern hardware combined with several new techniques for improving the method. A big reason for this is that Voxel Cone Tracing as shown in this thesis can provide realistic lighting with real time speeds for relatively complex scenes with several interesting properties. The capability to easily calculate several light bounces and have a method whose computational cost is mostly fixed to the amount of voxels used instead of the complexity of the scene is extremely useful in many cases. With further improvements Voxel Cone Tracing could prove to be extremely useful in the future, especially with less concern for the memory consumption of the method.

Voxel Cone Tracing is especially worth investigating more in the future for certain types of games where there are smaller scenes with a lot of change to really use Voxel Cone Tracings' strengths. Overall Voxel Cone Tracing is a technique which warrants more research and usage. It has, as previously noted, gained more traction again in later years as hardware has improved, and hopefully one day Voxel Cone Tracings' full potential of beautiful and fast lighting can be implemented in games or other interactive media. For now, it is nonetheless an incredibly interesting method with many unique properties worth investigating further in the future.

7 Sources

- [1] C. Crassin, F. Neyret, M. Sainz, S. Green, and E. Eisemann, "Interactive Indirect Illumination Using Voxel Cone Tracing," in *Proc. Pacific Graphics*, vol. 30, no. 7, 2011. [Online]. Available: https://research.nvidia.com/sites/default/files/pubs/2011-09_Interactive-Indirect-Illumination/GIVoxels-pg2011-authors.pdf [Accessed: 28-04-2024].
- [2] NVIDIA, "NVIDIA VXGI 2.0.1" [Online]. Available: <https://developer.nvidia.com/vxgi> [Accessed: 15-07-2024].
- [3] L. Szirmay-Kalos, L. Szécsi, and M. Sbert, "GPU-Based Techniques for Global Illumination Effects," *Synthesis Lectures on Computer Graphics and Animation*, vol. 2, no. 1, Jan. 2008, DOI: 10.2200/S00107ED1V01Y200801CGR004. Publisher: Morgan & Claypool Publishers.
- [4] T. Ritschel, C. Dachsbacher, T. Grosch, and J. Kautz, "The State of the Art in Interactive Global Illumination," *Computer Graphics Forum*, vol. 0, no. 0, pp. 1-27, 1981. [Online]. Available: https://jankautz.com/publications/GISTAR_CGF12.pdf [Accessed: 15-05-2024].
- [5] D. Lin, M. Kettunen, B. Bitterli, J. Pantaleoni, C. Yuksel, and C. Wyman, "Generalized Resampled Importance Sampling: Foundations of ReSTIR," *ACM Transactions on Graphics (SIGGRAPH 2022)*, July 24, 2022. [Online]. Available: https://research.nvidia.com/publication/2022-07_generalized-resampled-importance-sampling-foundations-restir [Accessed: 16-05-2024].
- [6] D. Lin, M. Kettunen, B. Bitterli, J. Pantaleoni, C. Yuksel, and C. Wyman, "Generalized Resampled Importance Sampling (GRIS)," University of Utah, 2022. [Online]. Available: <https://graphics.cs.utah.edu/research/projects/gris/> [Accessed: 17-05-2024].
- [7] J. T. Kajiya, "The Rendering Equation," *SIGGRAPH Comput. Graph.*, vol. 20, no. 4, pp. 143-150, Aug. 1986. doi: 10.1145/15886.15902. [Online]. Available: <http://doi.acm.org/10.1145/15886.15902> [Accessed: 17-05-2024].
- [8] NVIDIA, "Nsight-Graphics," Nvidia-Developer, [Online]. Available: <https://developer.nvidia.com/nsight-graphics>. [Accessed: June 4, 2024].
- [9] Blender Manual, "Introduction to Cycles," Blender Documentation, 2021. [Online]. Available: <https://docs.blender.org/manual/en/2.91/render/cycles/introduction.html>. [Accessed: June 4, 2024].
- [10] S. Green, "Chapter 16. Real-Time Approximations to Subsurface," NVIDIA Developer, GPU Gems [Online]. Available: <https://developer.nvidia.com/gpugems/gpugems/part-iii-materials/chapter-16-real-time-approximations-subsurface-scattering> [Accessed: 20-05-2025].
- [11] G. J. Ward, "Measuring and modeling anisotropic reflection," in *Proc. of SIGGRAPH '92*, 1992, pp. 265-272. [Online]. Available: <https://cseweb.ucsd.edu/~ravir/6998/papers/p265-ward.pdf> [Accessed: 17-05-2024].

- [12] A. Yuille, "Lecture 10: Statistical Models of Shape," Johns Hopkins University, Winter 2012. [Online]. Available: <https://www.cs.jhu.edu/~ayuille/courses/Stat238-Winter12/lecture10.pdf> [Accessed:17-05-2024].
- [13] J. de Vries, "Advanced Lighting," LearnOpenGL, [Online]. Available: <https://learnopengl.com/Advanced-Lighting/Advanced-Lighting> [Accessed: 06-05-2024].
- [14] P. Collingridge, "Rendering," The Science Behind PIXAR, 2017. [Online]. Available: <https://sciencebehindpixar.org/pipeline/rendering> [Accessed: 01-06-2024]
- [15] "NVIDIA Developer - The Basics of GPU Voxelization," NVIDIA, 22 March 2015. [Online]. Available: <https://developer.nvidia.com/content/basics-gpu-voxelization> [Accessed 12 April 2024].
- [16] R. Rauwendaal, "Voxel Based Indirect Illumination Using Spherical Harmonics," Ph.D. dissertation, Dept. [Dept. Name], [Univ. Name], [Location], 2013. Advisor: M. Bailey. DOI: 10.13140/2.1.1300.6403.
- [17] "glTF-Sample-Models," Khronos Group, GitHub, [Online]. Available: <https://github.com/KhronosGroup/glTF-Sample-Models/tree/main/2.0/Sponza> [Accessed: 04-04-2024].
- [18] A. Lingtorp, "Real Time Voxel Cone Tracing using Bilateral Filters and 3D Clipmaps," Thesis, KTH Royal Institute of Technology, 2020. [Online]. Available: <https://kth.diva-portal.org/smash/get/diva2:1440989/FULLTEXT01.pdf> [Accessed: 04-05-2024].
- [19] F. Losasso and H. Hoppe, "Geometry clipmaps: terrain rendering using nested regular grids," *ACM Transactions on Graphics*, vol. 23, no. 3, pp. 769–776, Aug. 2004. DOI: 10.1145/1015706.1015799.
- [20] "Specular vs. Diffuse Reflection," Wikimedia Commons. [Online]. Available: https://commons.wikimedia.org/wiki/File:Specular_v_Diffuse_reflection.png [Accessed: 21-05-2024].
- [21] "Rasterization of triangle" Scratchapixel, in "Rasterization: The Rasterization Stage." [Online]. Available: <https://www.scratchapixel.com/lessons/3d-basic-rendering/rasterization-practical-implementation/rasterization-stage.html> [Accessed: 21-05-2024].
- [22] K. Zsolnai-Fehér, "Ray Tracing: How NVIDIA Solved the Impossible!," *NVIDIA On-Demand, GTC Digital September*, 2022. [Online]. Available: <https://www.nvidia.com/en-us/on-demand/session/gtcfall22-a41171/> [Accessed: 16-05-2024].
- [23] Y. Zhou, H. Lu, G. Wang, J. Wang, and W. Li, "Voxelization modeling based finite element simulation and process parameter optimization for Fused Filament Fabrication," *Materials and Design*, [Online]. Available: <https://doi.org/10.1016/j.matdes.2019.108409> [Accessed: 16-05-2024].
- [24] C. Wyman and A. Panteleev, "Rearchitecting Spatiotemporal Resampling for Production," in *Proc. ACM/EG Symposium on High Performance Graphics*, July 6, 2021. [Online].

Available: https://research.nvidia.com/publication/2021-07_rearchitecting-spatiotemporal-resampling-production [Accessed: 16-05-2025].

[25] S. Marschner and P. Shirley, Fundamentals of Computer Graphics, 4th ed. Boca Raton, FL: A K Peters/CRC Press, 2018.

[26] M. Teschner, "Image Processing and Computer Graphics: Rendering Pipeline," Image Processing and Computer Graphics, University of Freiburg, [Online]. Available: https://cg.informatik.uni-freiburg.de/course_notes/graphics_01_pipeline.pdf [Accessed: 01-05-2024].

[27] B. T. Phong, Illumination for computer generated pictures, vol. 18, no. 6. New York, NY, USA: ACM, 1975, pp. 311–317. doi: 10.1145/360825.360839.

[28] Nvidia, "What is Denoising?," Nvidia Blog, May 2, 2018. [Online]. Available: <https://blogs.nvidia.com/blog/what-is-denoising/>. [Accessed: June 7, 2024].

[29] Khronos Group, "Compute Shader," OpenGL Wiki, [Online]. Available: https://www.khronos.org/opengl/wiki/Compute_Shader. [Accessed: June 2, 2024].

[30] Khronos Group, "ARB_sparse_texture," OpenGL Extensions, [Online]. Available: https://registry.khronos.org/OpenGL/extensions/ARB/ARB_sparse_texture.txt. [Accessed: may 10, 2024].

[31] Khronos Group, "NV_conservative_raster," OpenGL Extensions, [Online]. Available: https://registry.khronos.org/OpenGL/extensions/NV/NV_conservative_raster.txt. [Accessed: June 2, 2024].

[32] Khronos Group, "Texture," OpenGL Wiki, [Online]. Available: <https://www.khronos.org/opengl/wiki/Texture>. [Accessed: may 20, 2024].

Appendix

Cones used for implementation. Each cone is 60 degrees wide.

```
vec3 coneDirections[6] = vec3[]
(
    vec3(0, 1, 0),
    vec3(0, 0.5, 0.866025),
    vec3(0.823639, 0.5, 0.267617),
    vec3(0.509037, 0.5, -0.700629),
    vec3(-0.509037, 0.5, -0.700629),
    vec3(-0.823639, 0.5, 0.267617)
);
float coneWeights[6] = float[](0.25, 0.15, 0.15, 0.15, 0.15, 0.15);
```

